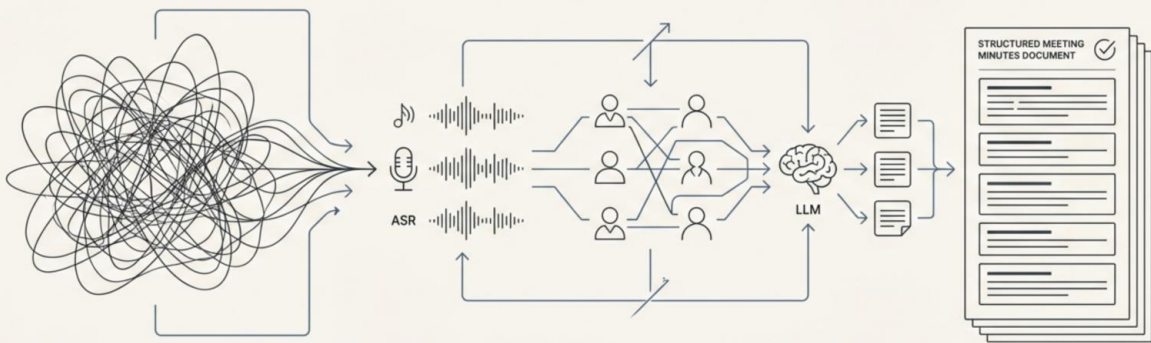


Architectural Blueprint

회의록을 정리하는 AI 서기: End-to-End 자동화 파이프라인 구축의 청사진

음성 인식(ASR), 화자 분리(Diarization), LLM을 결합한 포괄적 기술 분석 및 구현 가이드

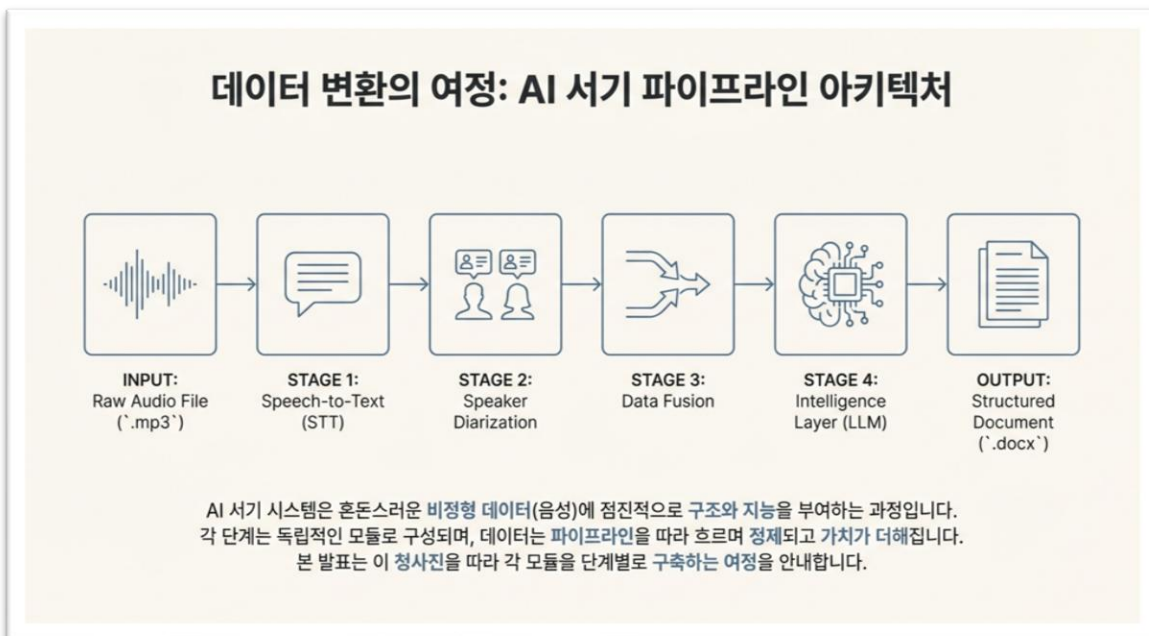


현대 조직의 핵심 의사결정 과정인 회의. 그러나 회의록 작성은 여전히 막대한 자원을 소모하는 비효율의 영역으로 남아있습니다. 본 가이드는 단순한 코드 구현을 넘어, 각 AI 모듈의 원리부터 기술적 난관 해결까지, 하나의 완성된 '지능형 시스템'을 구축하는 설계도를 제시합니다.

1. 서론: 비즈니스 자동화와 음성 기술의 융합

현대 조직 환경에서 회의는 의사결정과 정보 공유의 핵심 메커니즘으로 기능하지만, 그 내용을 기록하고 정리하는 과정은 여전히 막대한 인적 자원을 소모하는 비효율적인 업무로 남아 있습니다. "회의록을 정리하는 AI 서기" 프로젝트는 이러한 문제를 해결하기 위해 최신 인공지능 기술인 ASR(Automatic Speech Recognition, 자동 음성 인식), 화자 분리(Speaker Diarization), 그리고 거대 언어 모델(LLM)을 결합하여 종단간(End-to-End) 자동화 파이프라인을 구축하는 것을 목표로 합니다.

각 기술 모듈의 작동 원리, 아키텍처 설계의 타당성, 현업 적용 시 발생할 수 있는 기술적 난관(Dependency Conflict, Environment Setup 등)과 그 해결책을 심도 있게 분석합니다. 특히 클라우드 기반의 API 활용법과 로컬 환경에서의 오픈소스 모델 운용법을 비교 분석함으로써, 독자가 자신의 컴퓨팅 환경과 보안 요구사항에 최적화된 시스템을 설계할 수 있도록 돕는 데 주안점을 둡니다.



2. 음성을 텍스트로 변환하기 (STT): 클라우드 기반 접근법

STAGE 1 | 음성에 텍스트라는 목소리를 부여하다(STT)

모든 지능화의 시작은 음성을 정확한 텍스트로 변환하는 것입니다. OpenAI의 **Whisper** 모델은 68만 시간의 다국어 데이터를 학습하여, **트랜스포머 아키텍처** 기반으로 기존의 한계를 뛰어넘는 성능을 제공합니다.



클라우드 API: 속도, 편의성, 관리 효율성

OpenAI 등에서 제공하는 관리형 API를 활용합니다. 서버 구축 및 유지보수 없이 즉시 최고 수준의 성능을 사용할 수 있습니다.

- ✓ 높은 가용성
- ✓ 간편한 구현
- ✓ 초기 비용 없음.



로컬 환경: 보안, 통제권, 비용 최적화

민감 정보를 다루거나 대규모 데이터를 처리할 때, 자체 GPU 서버에 오픈소스 모델을 배포합니다.

- ✓ 데이터 프라이버시 보장
- ✓ 장기적 비용 절감 가능성
- ✓ 커스터마이징 용이.

2.1 ASR 기술의 진화와 OpenAI Whisper

과거의 음성 인식 기술은 통계적 모델(Hidden Markov Models)에 의존하여 잡음이 많은 환경이나 자연스러운 대화체 인식에 한계를 보였습니다. 그러나 트랜스포머(Transformer) 아키텍처의 등장과 함께, OpenAI의 Whisper 모델은 약 680,000 시간의 다국어 및 멀티태스크 감독 데이터를 학습하여 기존의 한계를 뛰어넘는 성능을 보여주고 있습니다.¹

본 프로젝트의 첫 번째 단계는 녹음된 회의 음성 파일을 텍스트 데이터로 변환하는 것입니다. 이를 위해 OpenAI가 제공하는 Whisper API를 활용합니다.

2.2 OpenAI Whisper API 활용 전략

2.2.1 비용 효율성 및 경제성 분석

기업 환경에서 API 도입 시 가장 중요한 고려 사항은 비용입니다. 2025 년 기준, OpenAI 의 Whisper API(whisper-1 모델) 가격 정책은 분당 0.006 달러로 책정되어 있습니다.

클라우드 접근법: Whisper API의 경제성과 보안 구현

비용 효율성 분석 (ROI)

1시간 분량의 회의를 처리하는 데 드는 비용은 약 500원 미만으로, 수 시간이 소요되는 인적 자원의 기회비용과 비교할 때 압도적인 ROI를 제공합니다.

서비스	모델	가격 (분당)	가격 (시간당)	비고
Whisper API	whisper-1	\$0.006	\$0.36	관리형 서비스, 높은 가용성
Deepgram	Nova-2	~\$0.0043	~\$0.26	배치 처리 기준
Azure OpenAI	Whisper	\$0.006	\$0.36	엔터프라이즈 보안 통합

보안 모범 사례 (Security Best Practice)

API 키는 코드에 하드코딩하지 않고, 'python-dotenv' 라이브러리를 통해 환경 변수('.env')로 관리하는 것이 필수입니다.

```
from dotenv import load_dotenv
import os

# 환경 변수 로드 (보안 강화)
load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
```

숨겨진 강력한 기능: 번역(Translation)

".translations.create" 메서드를 사용하면 다국어 회의 음성을 별도 번역 모델 없이 즉시 영어 텍스트로 변환할 수 있습니다. 이는 처리 시간과 토큰 비용을 이중으로 절감하는 효과적인 전략입니다.

위 표에서 볼 수 있듯이, Whisper API 는 경쟁 서비스 대비 합리적인 가격대를 형성하고 있습니다. 예를 들어, 1 시간 분량의 임원 회의를 전사 처리하더라도 비용은 약 500 원(환율 1,400 원 가정 시) 미만에 불과합니다. 이는 인적 자원이 투입되어 수 시간을 소요하는 비용(기회비용 포함)과 비교할 때 압도적인 ROI(투자 대비 효과)를 제공합니다.

2.2.2 파이썬 클라이언트 구현 및 보안 모범 사례

API 를 활용한 구현에서는 보안이 최우선 과제입니다. API 키를 코드 내에 하드코딩하는 것은 보안상 치명적인 취약점이 될 수 있으므로, python-dotenv 라이브러리를 통해 환경 변수(.env)로 관리하는 것이 필수적입니다.

sec01/penai_api_whisper.ipynb

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
```

음성 파일 처리 로직은 단순하지만 강력합니다. 바이너리 읽기 모드(rb)로 파일을 열어 API 로 전송하면, 서버 측의 고성능 GPU 클러스터가 추론을 수행하고 텍스트를 반환합니다.

```
audio_file_path = '../audio/lisy_audio_2023_58s.mp3' # MP3 파일 경로 입력
```

```
with open(audio_file_path, 'rb') as audio_file:
    transcription = client.audio.transcriptions.create(
        model="whisper-1",
        file=audio_file
    )
```

```
transcription
```

```
with open(audio_file_path, 'rb') as audio_file:
    transcription = client.audio.translations.create(
        model="whisper-1",
        file=audio_file,
    )
```

```
transcription
```

심층 분석: 번역(Translation) 기능의 활용

Whisper API의 강력한 기능 중 하나는 단순 전사(Transcription)를 넘어선 번역(Translation) 기능입니다. **.transcriptions.create** 대신 **.translations.create** 메서드를 호출하면, 입력된 오디오가 한국어, 일본어, 불어 등 어떤 언어이든 상관없이 영어 텍스트로 직접 변환되어 출력됩니다. 이는 다국적 회의에서 즉각적인 영문 회의록을 생성해야 할 때, 별도의 번역 모델을 거치지 않아도 되므로 토큰 비용과 처리 시간을 이중으로 절감하는 효과가 있습니다.

3. 로컬 환경에서의 음성 변환: 프라이버시와 성능의 균형

소스코드

- huggingface_whisper.ipynb : 허깅페이스 연동 실습
- speaker_diarization.ipynb : 화자 분리 실습
- whisper_stt.ipynb : STT 실습
- summarize_and_correct.ipynb : LLM 연동 실습

3.0 개발 환경 구축(중요)

개요 (Executive Summary)

현대 음성 처리(Speech Processing) 기술, 특히 화자 분리(Speaker Diarization) 분야는 단순한 신호 처리를 넘어 복잡한 신경망 아키텍처와 대규모 데이터 파이프라인의 통합을 요구한다. 본 보고서는 pyannote-audio 3.1.1 프레임워크를 중심으로, 최신 하드웨어 가속 기술(CUDA 12.1)과 윈도우(Windows) 운영체제의 특성을 고려한 최적의 개발 환경 구축 전략을 제시한다.

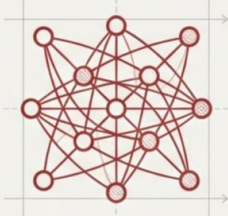
각 단계가 지니는 기술적 함의, 잠재적 종속성 충돌(Dependency Conflict), 그리고 시스템 레벨의 동적 라이브러리(DLL) 관리 메커니즘을 심층적으로 분석한다. 특히, 데이터 과학 분야에서 널리 쓰이는 Conda 패키지 관리자를 배제하고, Python의 순수 가상 환경(venv)과 pip만을 사용하여 재현 가능하고 독립적인 환경을 구축하는 "Pure Python" 접근 방식을 채택한다. 이는 상용 배포 환경이나 제한된 서버 환경에서의 이식성을 극대화하기 위한 전략적 선택이다.

전반에 걸쳐 numpy 2.0 ABI(Application Binary Interface) 호환성 문제, torchcodec 과 torchaudio 간의 백엔드 충돌, 그리고 윈도우 보안 모델 변경에 따른 FFMPEG DLL 로딩 문제 등 실무에서 직면할 수 있는 치명적인 기술적 난제들을 해결하는 이론적 배경과 실질적인 해결책을 상세히 기술한다.

왜 윈도우 AI 환경 구축은 실패하는가? (The Minefield)

단순한 `pip install`의 반복은 필연적으로 실패로 이어집니다.

성공은 숨겨진 기술적 난제, 즉 '지뢰밭'을 인지하고 전략적으로 회피하는 데에 달려있습니다.



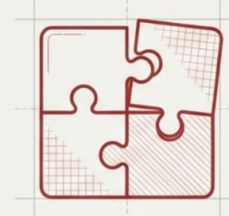
의존성 지옥 (Dependency Hell)

Numpy 2.0 ABI 호환성 파괴,
torch와 torchaudio 버전 불일치 등,
패키지 간의 미세한 충돌이 전체
시스템을 붕괴시킵니다.



DLL 충돌 및 로딩 실패 (DLL Conflicts)

Python 3.8+의 변경된 보안 정책으로
인해 FFmpeg 등의 핵심 라이브러리를
시스템이 찾지 못하고 프로세스가
즉시 충돌합니다.



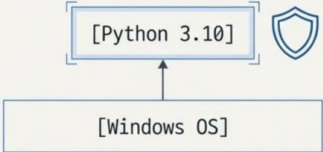
바이너리 비호환성 (Binary Incompatibility)

CPU 전용 PyTorch가 설치되거나,
시스템의 NVIDIA 드라이버와 CUDA
버전이 맞지 않아 GPU 가속을 전혀
활용하지 못하는 문제가 발생합니다.

3.1. 기반 런타임 환경 설계 및 격리 전략

음성 인식 및 화자 분리 파이프라인의 안정성은 견고한 기초 런타임 환경에서 시작된다. 단계 0 부터 2 까지의 과정은 단순한 설치를 넘어, 시스템의 오염을 방지하고 프로젝트별 독립성을 보장하기 위한 격리 아키텍처를 수립하는 과정이다.

첫 번째 전략: 순수하고 격리된 기반 다지기



Step 0: Python 3.10 설치
[\[Link to Python 3.10 download\]](#)
설치 시 반드시 'Add Python 3.10 to PATH' 옵션을 체크하십시오.

Step 1: 가상 환경 생성

```
python -m venv venv
```

Python 3.10 선택의 이유: 최신 버전이 아닌 3.10을 선택하는 것은 안정성과 호환성의 '황금률(Golden Mean)'을 찾기 위함입니다.

- **안정성:** 'PyTorch', 'pyannote'의 핵심 C-extension 의존성 패키지들이 가장 안정적으로 바이너리 휠(Wheel)을 지원하는 버전입니다.
- **위험 회피:** Python 3.12+의 'distutils' 제거와 같은 큰 변화로 인한 컴파일 오류 가능성을 원천 차단합니다.

'venv'를 통한 완벽한 격리:

- Conda와 달리, venv는 Python 패키지만을 격리하여 시스템 의존성(FFMPEG, CUDA)에 대한 **명시적인 제어권**을 사용자에게 부여합니다. 이는 문제 발생 시 원인 파악을 용이하게 합니다.

Python 3.10 런타임의 기술적 타당성 (단계 0)

프로토콜: python=3.10 개발 환경 구성

현재 Python 생태계는 3.13 버전까지 출시되었으나, 본 연구 환경에서 Python 3.10 을 채택한 것은 안정성과 호환성 사이의 "Golden Mean(중용)"을 찾기 위함이다. 딥러닝 프레임워크, 특히 pyannote-audio 와 같은 특수 목적 라이브러리는 핵심 의존성 패키지(PyTorch, Numpy, C-extension 기반 오디오 처리기 등)의 바이너리 휠(Wheel) 지원 여부에 따라 사용 가능한 Python 버전이 제한된다.¹

- **호환성 매트릭스 분석:** pyannote-audio 3.1.1 은 PyTorch 2.x 생태계 위에서 작동하며, 이는 Python 3.8 이상 3.11 이하의 환경에서 가장 안정적인 동작을 보장한다. Python 3.12 이상의 환경에서는 distutils 의 제거와 같은 표준 라이브러리의 대대적인 변경으로 인해 레거시 빌드 도구에 의존하는 일부 오디오 처리 라이브러리 설치 시 컴파일 오류가 발생할 위험이 높다.
- **성능 고려:** Python 3.10 은 3.9 대비 구조적 패턴 매칭(Structural Pattern Matching) 도입 등 문법적 효율성이 개선되었으며, CPython 인터프리터의 최적화를 통해 대규모 오디오 데이터 로딩 시 오버헤드를 줄이는 데 기여한다.

윈도우 환경에서의 Python 3.10 설치 시 반드시 시스템 PATH 환경 변수에 인터프리터를 등록하여, 커맨드 라인 인터페이스(CLI)에서 전역적인 접근이 가능하도록 구성해야 한다. 이는 후술할 venv 모듈 호출의 전제 조건이 된다.

1.2 가상화 아키텍처: venv 를 통한 의존성 격리 (단계 1)

프로토콜: `python -m venv venv`

Conda 를 배제한 상황에서 Python 표준 라이브러리인 venv 모듈의 사용은 필수적이다. venv 는 "경량화된 가상 환경"을 지향하며, 프로젝트 디렉터리 내에 독립적인 Python 실행 파일과 라이브러리 저장소(site-packages)를 생성한다.

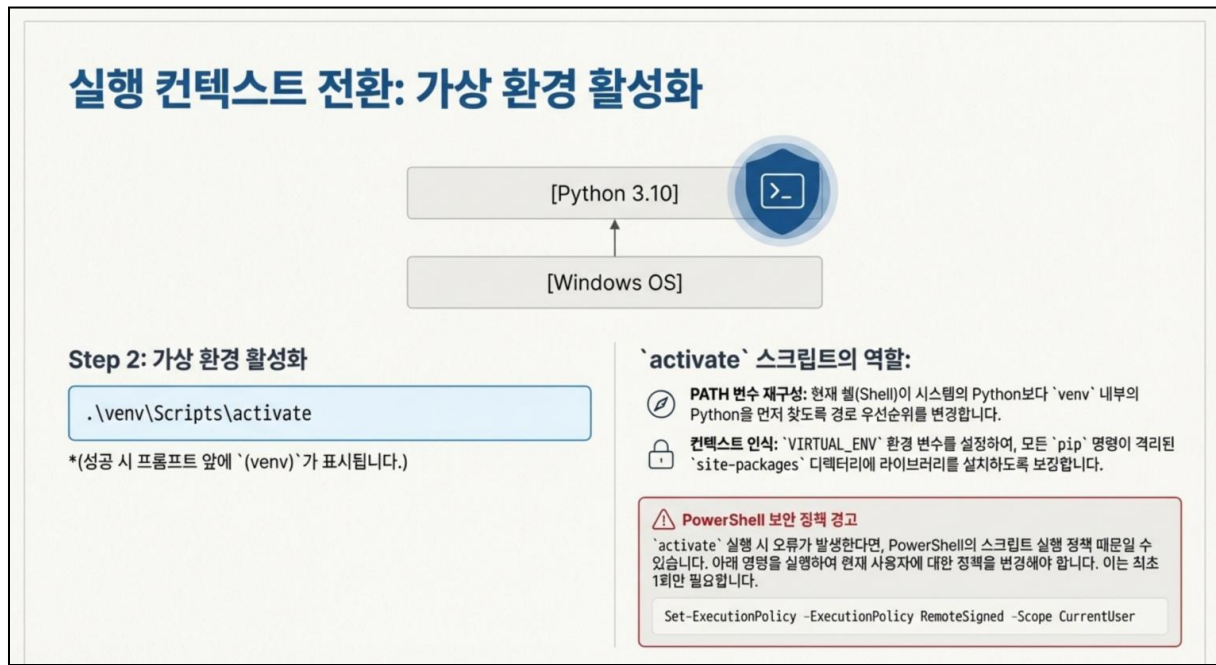
1.2.1 venv 의 작동 메커니즘과 Conda 와의 차이점

Conda 가 운영체제 레벨의 바이너리(예: CUDA Toolkit, FFMPEG 바이너리 자체)까지 관리하는 포괄적인 시스템 패키지 관리자라면, venv 는 철저히 Python 패키지 인덱스(PyPI) 기반의 라이브러리만을 격리한다.

특성	venv (Virtual Environment)	Conda
관리 대상	Python 패키지 (PyPI)	Python 패키지 + 시스템 바이너리 (C/C++ 라이브러리)
격리 수준	Python 인터프리터 및 라이브러리 경로	독립적인 실행 환경 전체 (가상 머신에 가까움)
Windows 통합	시스템 DLL 의존성 수동 관리 필요	바이너리 설치 시 DLL 경로 자동 처리
사용 사례	가볍고 표준화된 배포, Docker 컨테이너	데이터 과학 분석, 복잡한 비 Python 의존성 관리

본 프로젝트에서 venv 를 선택함으로써 얻는 이점은 **명시적인 제어권**이다. 사용자는 FFMPEG 이나 CUDA 와 같은 외부 바이너리 의존성을 시스템 레벨에서 직접 관리하게 되며, 이는 "블랙박스"적인 Conda 의 동작 방식보다 문제 해결(Troubleshooting) 시 명확한 원인 파악을 가능하게 한다.

1.2.2 환경 활성화 및 실행 컨텍스트 전환 (단계 2)



```
.\venv\Scripts\activate
```

```
python -m pip install --upgrade pip (반드시 진행 필요)
```

가상 환경의 생성만으로는 충분하지 않으며, 현재 셸(Shell)의 실행 컨텍스트를 해당 환경으로 전환해야 한다. 윈도우에서 `activate` 스크립트의 실행은 다음과 같은 환경 변수 조작을 수행한다:

- PATH 변수 재구성:** 시스템의 기본 Python 경로보다 `venv\Scripts` 경로를 우선순위로 배치한다. 이를 통해 `python` 또는 `pip` 명령 실행 시 전역 설치 버전이 아닌 가상 환경 내부의 실행 파일이 호출된다.
- VIRTUAL_ENV 설정:** 현재 활성화된 가상 환경의 루트 경로를 환경 변수에 등록하여, 하위 프로세스들이 격리된 환경을 인식하도록 한다.

기술적 주의사항 (Windows PowerShell 보안 정책):

윈도우 PowerShell 환경에서 스크립트 실행 정책(Execution Policy)이 기본값(Restricted)으로 설정되어 있을 경우, 활성화 스크립트 실행이 차단될 수 있다. 이를 해결하기 위해 다음과 같은 전처리 명령이 필요할 수 있다. 이는 개발 환경 구성 시 흔히 간과되는 보안 레이어이다.

PowerShell

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

2. 유틸리티 레이어 및 데이터 파이프라인 구성

격리된 환경 내부에서, 본격적인 딥러닝 모델 구동에 앞서 데이터 전처리, 시각화, 그리고 외부 API 연동을 담당하는 지원 소프트웨어 스택을 구축한다.

파이프라인 확장성 및 개발 편의성 확보

```
graph LR; A["[Python 3.10]  
active virtual environment  
[Windows OS]"] --> B["[OpenAI]"]; A --> C["[Jupyter]"]; A --> D["[DotEnv]"];
```

Step 3 & 4: 핵심 유틸리티 및 Hugging Face 생태계 통합

```
pip install openai ipykernel dotenv matplotlib  
pip install transformers datasets[audio] accelerate
```

이유:

- **openai**: 화자 분리 결과를 LLM에 주입하여 요약, 역할 분석 등 **GenAI 워크플로우로 확장**하기 위한 필수 인터페이스입니다.
- **ipykernel & matplotlib**: VS Code나 Jupyter에서 오디오 파형, 화자 분리 결과(Gantt Chart)를 **실시간으로 시각화**하고 검증하는 대화형 개발 환경을 구축합니다.
- **'python-dotenv'**: Hugging Face 토큰과 같은 민감한 정보를 **.env** 파일로 분리하여 코드로부터 보호하는 **보안 모범 사례**입니다.
- **'datasets[audio]'**: 대규모 오디오 데이터셋을 효율적으로 처리하고, FFmpeg 설정 실패 시 **백업 오디오 디코더** 역할을 수행합니다.

2.1 연구 지원 및 시각화 도구 (단계 3)

프로토콜: `pip install openai ipykernel dotenv matplotlib python-docx`

이 단계에서 설치되는 라이브러리들은 화자 분리 파이프라인의 전후 처리를 담당하는 핵심 유틸리티이다.

- **openai**: 최신 화자 분리 파이프라인은 단순히 "누가 언제 말했는가"를 식별하는 데 그치지 않고, 식별된 화자의 발화 내용을 텍스트로 변환(STT)한 후 이를 거대언어모델(LLM)에 주입하여 요약, 감정 분석, 역할 부여 등을 수행하는 "GenAI" 워크플로우로 확장된다. openai 라이브러리는 이러한 확장성을 위한 인터페이스를 제공한다.
- **ipykernel**: 개발 환경의 상호작용성(Interactivity)을 보장한다. Jupyter Notebook 이나 VS Code 의 대화형 창에서 현재의 venv 를 커널로 인식하게 하여, 오디오 파형을 실시간으로 시각화하거나 모델의 출력을 단계별로 검증할 수 있게 한다.
- **python-dotenv**: ****보안 모범 사례(Security Best Practices)****의 구현이다. pyannote-audio 의 사전 학습된 모델은 Hugging Face Hub 의 인증을 요구한다.² 사용자 액세스 토큰과 같은

민감한 정보를 소스 코드에 하드코딩하는 대신, 환경 변수 파일(.env)에서 동적으로 로드하여 보안 사고를 예방한다.

- **matplotlib**: 오디오 신호 처리에서 시각화는 필수적이다. 화자 분리 결과(RTTM 포맷)를 간트 차트(Gantt Chart) 형태로 시각화하거나, 오디오의 멜 스펙트로그램(Mel-Spectrogram)을 그려 데이터의 품질을 육안으로 검증하는 데 사용된다.

2.2 허깅페이스(Hugging Face) 생태계 통합 (단계 4)

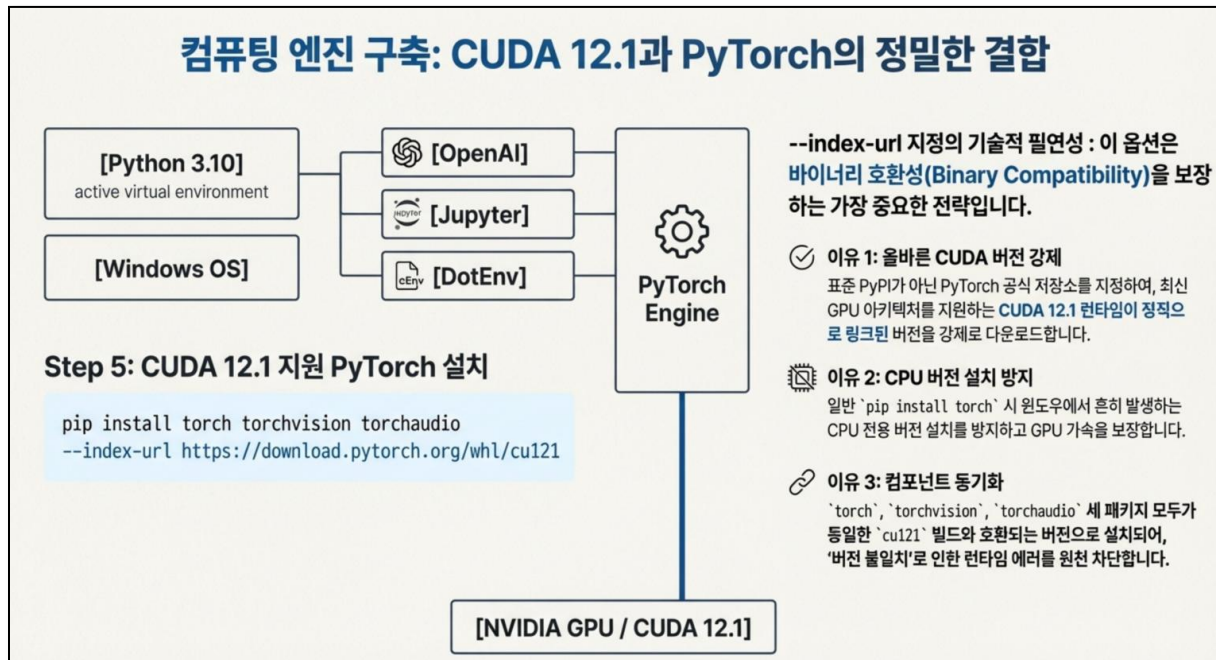
프로토콜: `pip install transformers datasets[audio] accelerate`

현대 자연어 처리 및 오디오 처리의 사실상 표준(De facto Standard)인 Hugging Face 생태계를 통합한다.

- **transformers**: pyannote 자체는 독자적인 모델 구조를 가지나, 종종 화자 분리 후처리 단계에서 Whisper(OpenAI)나 Wav2Vec2 와 같은 ASR 모델과의 결합이 요구된다. transformers 라이브러리는 이러한 모델들을 즉시 호출할 수 있는 통합 인터페이스를 제공한다.
- **datasets[audio]**: 대규모 오디오 데이터셋(LibriSpeech, VoxCeleb 등)을 효율적으로 로딩하고 스트리밍할 수 있는 라이브러리이다. 특히 [audio] 옵션(extras_require)을 명시함으로써, 오디오 디코딩에 필요한 soundfile, librosa 등의 하위 의존성을 함께 설치한다.⁴ 이는 추후 FFMPEG 설정 실패 시 백업 디코더로서의 역할도 수행한다.
- **accelerate**: PyTorch 모델의 하드웨어 가속을 추상화한다. 복잡한 멀티 GPU 설정이나 혼합 정밀도(Mixed Precision, FP16) 연산을 코드 변경 없이 적용할 수 있게 해주며, 특히 VRAM 소모가 큰 화자 임베딩 추출 과정에서 메모리 효율성을 최적화하는 데 기여한다.

3. 딥러닝 컴퓨팅 코어 구축: PyTorch 와 CUDA

본 개발 환경의 심장부인 딥러닝 프레임워크를 설치하는 단계이다. 윈도우 환경에서 GPU 가속을 활성화하기 위해서는 매우 정교한 버전 관리가 요구된다.



3.1 CUDA 12.1 기반 PyTorch 생태계 구축 (단계 5)

프로토콜: `pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121`

사용자는 표준 PyPI 저장소가 아닌, PyTorch 공식 휠 저장소(download.pytorch.org/whl/cu121)를 명시적으로 지정(--index-url)하였다. 이는 딥러닝 환경 구축에서 가장 중요한 **"바이너리 호환성(Binary Compatibility)"** 확보 전략이다.

3.1.1 --index-url 지정의 기술적 필연성

pip install torch 를 인자 없이 실행할 경우, pip 는 호스트 OS 에 가장 적합하다고 판단되는 패키지를 PyPI 에서 다운로드한다. 그러나 PyPI 에 업로드된 기본 torch 패키지는 종종 구버전 CUDA 런타임(예: 11.7, 11.8)을 포함하거나, 윈도우의 경우 CPU 전용 버전이 우선순위로 잡히는 경우가 있다.

명시적으로 cu121 인덱스를 지정함으로써, 사용자는 NVIDIA 의 최신 아키텍처(Ada Lovelace, Hopper 등)를 지원하는 CUDA 12.1 런타임이 정적으로 링크된 torch 바이너리를 강제로 설치하게 된다. 이는 로컬 시스템에 설치된 NVIDIA 드라이버 버전(530.xx 이상 권장)과 일치해야

하며, 최신 GPU의 성능을 온전히 이끌어내는 데 필수적이다.⁵

3.1.2 컴포넌트 간 동기화

이 명령어는 torch(텐서 연산 코어), torchvision(이미지 처리), torchaudio(오디오 신호 처리) 세 가지 패키지를 동시에 설치한다. 중요한 점은 이 세 패키지가 모두 동일한 CUDA 버전(12.1)과 호환되도록 빌드된 버전을 가져온다는 것이다. 만약 이들을 개별적으로 설치하거나 인덱스를 누락할 경우, torch는 CUDA 12.1 버전인데 torchaudio는 CPU 버전이 설치되는 등의 "버전 불일치(Version Mismatch)"가 발생하여 런타임 에러(RuntimeError: CUDA not available)를 유발할 수 있다.⁶

3.1.3 pyannote-audio와의 상호작용

pyannote-audio는 오디오 로딩 및 전처리를 위해 torchaudio에 강하게 의존한다. 특히 pyannote-audio 3.1.1은 torchaudio 2.x대 버전을 요구하는데, 여기서 윈도우 환경 특유의 심각한 백엔드 충돌 문제가 발생한다. 이는 다음 단계인 torchcodec 제거의 원인이 된다.

4. 의존성 충돌 해결 및 시스템 안정화

최신 Python 과학 생태계는 과도기적 단계에 있으며, 특히 2024-2025 년 시점에서 **Numpy 2.0**의 출시와 **PyTorch 오디오 백엔드의 개편**으로 인해 수많은 라이브러리 간 충돌이 보고되고 있다. 단계 6 과 8 은 이러한 "Dependency Hell(의존성 지옥)"을 탈출하기 위한 외과 수술적 조치이다.

첫 번째 지뢰 제거: Windows DLL 충돌을 유발하는 `torchcodec`

Step 6: torchcodec 패키지 제거 (Negative Step)

```
pip uninstall torchcodec
```

이유:

- **'torchcodec'의 문제점:** 'torchaudio'의 최신 의존성인 'torchcodec'은 고성능을 위해 FFMPEG DLL을 직접 동적 로딩(Dynamic Loading)하려 시도합니다.
- **Windows에서의 실패 원인:** Python 3.8 이후 변경된 윈도우의 **DLL 검색 경로(DLL Search Order)** 보안 정책으로 인해, 'torchcodec'은 시스템에 FFMPEG이 존재하더라도 이를 찾지 못하고 `RuntimeError: Could not load libtorchcodec...` 오류와 함께 프로세스를 즉시 충돌시킵니다.
- **우리의 전략:** 'torchcodec'을 의도적으로 제거하여, 'torchaudio'가 SoundFile과 같은 안정적인 레거시(Legacy) 백엔드로 폴백(Fallback)하도록 강제합니다. 이는 안정성을 위해 최첨단 기능을 포기하는 전략적 선택입니다.

4.1 TorchCodec 제거 및 오디오 백엔드 재정렬 (단계 6)

프로토콜: `pip uninstall torchcodec`

이 단계는 직관적이지 않으나, 윈도우 환경에서 pyannote-audio 를 구동하기 위해 필수적인 "Negative Step(제거 단계)"이다.

4.1.1 TorchCodec 의 역할과 윈도우에서의 실패

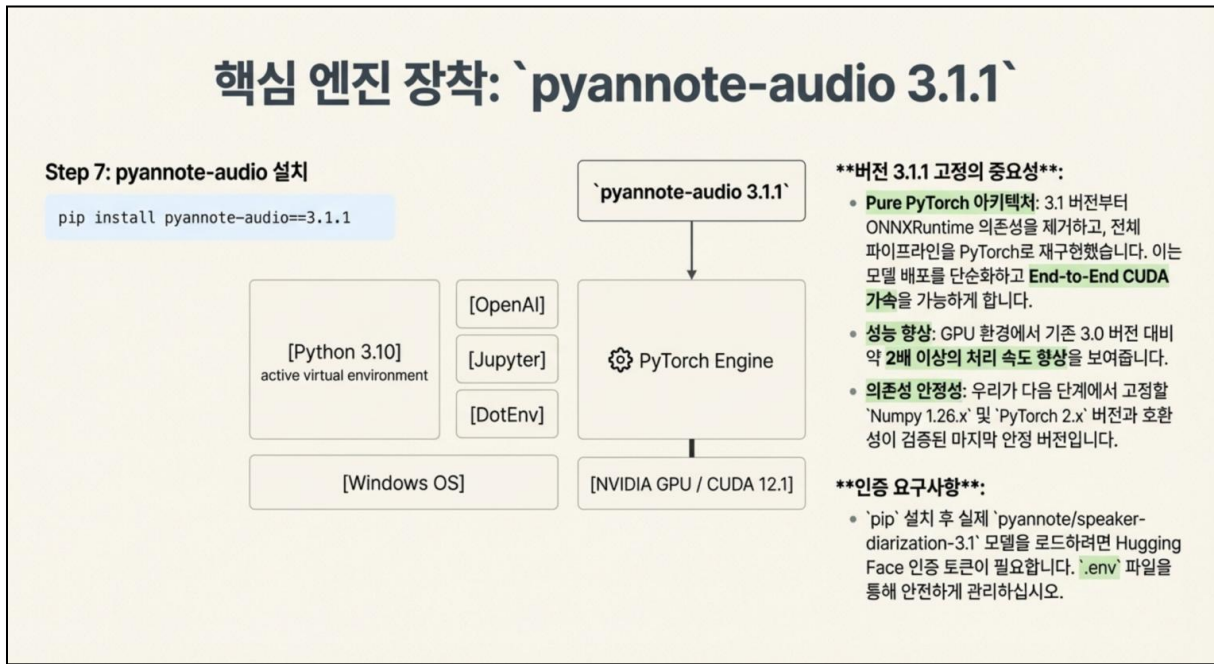
torchcodec 은 Meta(Facebook)가 개발한 차세대 미디어 디코딩 라이브러리로, torchaudio 최신 버전 설치 시 의존성으로 함께 설치되는 경우가 많다. 이 라이브러리는 성능 극대화를 위해 FFMPEG 라이브러리를 직접 동적 로딩(Dynamic Loading)하여 사용한다.

문제는 윈도우 환경에서의 DLL 검색 경로(DLL Search Order) 이슈이다. torchcodec 은 시스템에 설치된 FFMPEG 의 DLL(avcodec-*.dll, avutil-*.dll 등)을 찾으려 시도하는데, Python 3.8 이후 변경된 윈도우 DLL 로딩 보안 정책으로 인해 시스템 PATH 에 FFMPEG 이 존재하더라도 이를 찾지 못하고 프로세스가 즉시 충돌(Crash)한다.⁷

- **오류 현상:** `RuntimeError: Could not load libtorchcodec. Likely causes: Ffmpeg is not properly installed...`

- **해결 전략:** torchcodec 을 제거함으로써 torchaudio 가 기본 백엔드(SoundFile)나 FFMPEG 서브프로세스 호출 방식과 같은 레거시(Legacy) 하지만 안정적인 방식으로 폴백(Fallback)하도록 강제한다. 이는 시스템의 최첨단 기능 일부를 포기하는 대신 전체 파이프라인의 실행 가능성을 확보하는 전략이다.

4.2 화자 분리 엔진 설치 (단계 7)

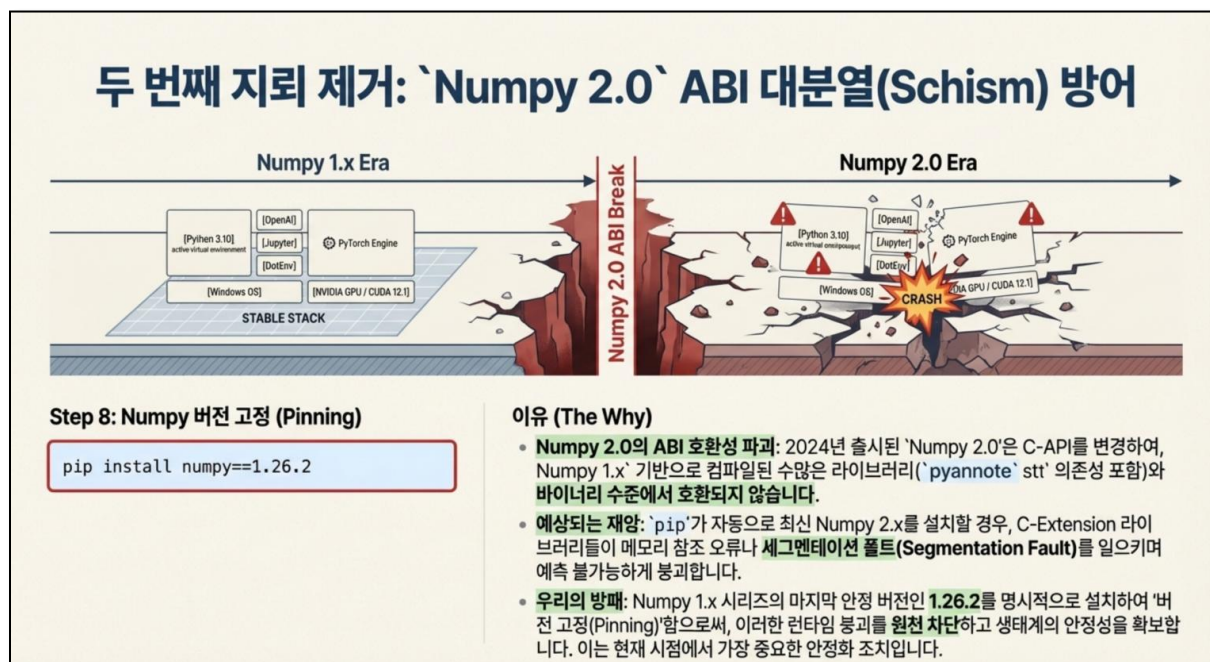


프로토콜: pip install pyannote-audio==3.1.1

핵심 라이브러리인 pyannote-audio 버전을 3.1.1 로 고정하여 설치한다.

- **아키텍처 변화:** 3.1 버전은 기존 3.0 버전에서 사용하던 ONNXRuntime 의존성을 제거하고, 전체 파이프라인(화자 분할 및 임베딩 추출)을 **Pure PyTorch** 로 재구현하였다.² 이는 모델 배포의 복잡성을 낮추고, PyTorch 의 CUDA 가속을 끊임 없이(End-to-End) 활용할 수 있게 하여 추론 속도를 향상시킨다.
- **성능 지표:** 3.1.1 버전은 3.0 대비 약 2 배 이상의 처리 속도 향상을 보여주며, 특히 GPU 환경에서 효율적이다.
- **인증 요구사항:** 설치 자체는 pip 로 가능하나, 실제 모델(pyannote/speaker-diarization-3.1) 로딩 시 Hugging Face 의 인증 토큰이 필요하다는 점을 명심해야 한다. 이는 단계 3 에서 설치한 dotenv 를 통해 관리된다.

4.3 Numpy 버전 고정을 통한 ABI 충돌 방지 (단계 8)



프로토콜: `pip install numpy==1.26.2`

이 단계는 현재 시점에서 가장 중요한 안정성 확보 조치 중 하나이다.

4.3.1 Numpy 2.0 ABI Schism (대분열)

2024 년 중반, numpy 2.0 이 출시되면서 C-API 수준에서의 큰 변화가 있었다(ABI 호환성 파괴). 이로 인해 numpy 1.x 버전을 기준으로 컴파일된 수많은 C-Extension 라이브러리들이 numpy 2.0 환경에서 실행될 경우 메모리 참조 오류나 세그멘테이션 폴트(Segmentation Fault)를 일으키며 붕괴하는 현상이 발생했다.¹

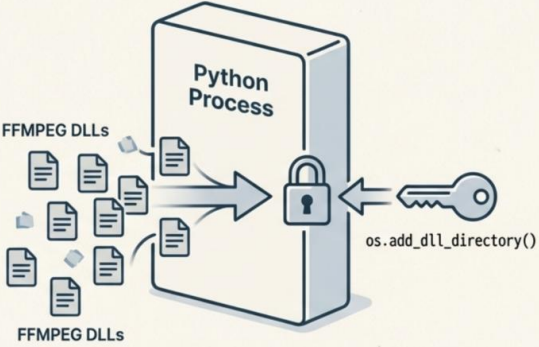
4.3.2 Pyannote 생태계와의 충돌

pyannote-audio 가 의존하는 pyannote-core, pyannote-metrics 등의 하위 라이브러리들은 특정 버전(>=2.0)의 numpy 를 요구하거나, 혹은 반대로 구버전 numpy 에 최적화되어 있는 등 의존성 트리가 복잡하다.¹¹ pip 의 의존성 해결자(Resolver)는 종종 최신 버전인 numpy 2.x 를 설치하려 시도하는데, 이는 pyannote-audio 3.1.1 및 torch 2.1/2.2 스택과 충돌을 일으킬 확률이 매우 높다. 따라서, **Numpy 1.x 시리즈의 최후반 안정 버전인 1.26.2** 를 명시적으로 설치하여 "Pinning(버전 고정)"함으로써, numpy 2.0 으로 인한 런타임 붕괴를 **완전 차단**하고 기존 생태계와의 완벽한 바이너리 호환성을 보장한다.

5. 윈도우 특화 시스템 프로그래밍: FFMPEG DLL 주입

Conda 를 사용하지 않는 "Pure Python" 환경에서 윈도우의 멀티미디어 처리를 구현하는 것은 시스템 프로그래밍 수준의 접근을 요구한다. 단계 9 는 이 난관을 극복하는 핵심 솔루션이다.

마스터 키: FFMPEG DLL을 Python 프로세스에 직접 주입하기



올바른 FFMPEG 빌드 선택
반드시 'Shared' 빌드를 다운로드해야 합니다. Static 빌드는 .exe 파일 하나로 구성되어 Python0이 라이브러리로 사용할 수 없습니다.
Gyan.dev 의 ffmpeg-release-full-shared.7z 버전을 권장합니다.

도전 과제
Python 3.8+ 윈도우는 보안 상의 이유로 PATH 환경변수에 있는 DLL을 자동으로 로드하지 않습니다.

해결책
코드 레벨에서 os.add_dll_directory() API를 사용하여 FFMPEG의 bin 폴더를 신뢰할 수 있는 경로로 직접 등록해야 합니다.

Step 9: FFMPEG DLL 로딩 코드를 스크립트 최상단에 추가

```
import os
import platform

# torchaudio나 pyannote 임포트 전에 실행되어야 합니다.
FFMPEG_BIN_PATH = r"C:\path\to\ffmpeg\bin" # 실제 경로로 수정

if platform.system() == 'Windows':
    try:
        os.add_dll_directory(FFMPEG_BIN_PATH)
        print("성공: FFMPEG DLL 경로가 주입되었습니다.")
    except (FileNotFoundError, AttributeError):
        print("경고: FFMPEG 경로를 찾을 수 없거나 DLL 주입에 실패했습니다.")
```

5.1 FFMPEG 빌드 유형의 선택: Shared vs. Static

가장 먼저 사용자는 적절한 FFMPEG 바이너리를 확보해야 한다. 여기서 "Shared(공유)" 빌드와 "Static(정적)" 빌드의 차이를 이해하는 것이 결정적이다.¹²

- **Static Build:** ffmpeg.exe 파일 하나에 모든 코덱 라이브러리가 포함되어 있다. 커맨드 라인 실행에는 편리하나, 다른 프로그램(Python)이 라이브러리(dll)를 빌려 쓰는 것은 불가능하다.
- **Shared Build:** ffmpeg.exe 는 껍데기에 불과하며, 실제 기능은 avcodec-60.dll, avformat-60.dll 등의 동적 라이브러리 파일로 분리되어 있다. torchaudio 나 pydub 과 같은 Python 라이브러리는 바로 이 DLL 파일들을 필요로 한다.

요구사항: 사용자는 반드시 **Shared Build** (예: Gyan.dev 의 ffmpeg-release-full-shared.7z)를 다운로드하여 압축을 해제해야 한다.

5.2 DLL 검색 경로 주입 코드 구현 (단계 9)

프로토콜: FFMPEG DLL 로딩 코드로 처리

Python 3.8 이상 윈도우 버전은 보안상의 이유로 PATH 환경 변수에 있는 DLL 을 자동으로 로드하지 않는다. 따라서 개발자가 코드 레벨에서 명시적으로 DLL 디렉터리를 "신뢰할 수 있는 경로"로 등록해야 한다.⁸

다음은 이 기능을 수행하는 검증된 파이썬 코드 블록이다. 이 코드는 pyannote 나 torch 를 임포트하기 전에 최상단에서 실행되어야 한다.

```
import os
import platform
import sys

def setup_ffmpeg_dll_path(ffmpeg_bin_path: str):
    """
    윈도우 환경에서 FFMPEG Shared DLL 을 Python 프로세스에 주입합니다.
    이 함수는 torchaudio 나 pyannote.audio 가 임포트되기 전에 실행되어야 합니다.
    """
    # 1. 운영체제 확인: 윈도우가 아니면 아무 작업도 하지 않음
    if platform.system() != 'Windows':
        return

    # 2. 경로 유효성 검사
    if not os.path.exists(ffmpeg_bin_path):
        raise FileNotFoundError(f"FFMPEG 경로를 찾을 수 없습니다: {ffmpeg_bin_path}\n"
                                f"반드시 'Shared' 빌드를 다운로드하여 해당 경로에 압축을 해제하십시오.")

    # 3. Python 3.8+ DLL 디렉터리 추가 (핵심 로직)
    # os.add_dll_directory 는 윈도우 전용 API 임
    try:
        os.add_dll_directory(ffmpeg_bin_path) # 핵심 코드
```

```

print(f" FFMPEG DLL 디렉터리가 로드되었습니다: {ffmpeg_bin_path}")
except AttributeError:
    # Python 3.8 미만 버전 호환성 (본 가이드는 3.10 이므로 실행될 일 없음)
    os.environ = ffmpeg_bin_path + os.pathsep + os.environ

# 사용 예시 (실제 사용자의 FFMPEG 설치 경로로 수정 필요)
# 주의: 'bin' 폴더 안에.dll 파일들이 존재해야 함
FFMPEG_DIR = r"C:\ffmpeg\bin"
setup_ffmpeg_dll_path(FFMPEG_DIR)

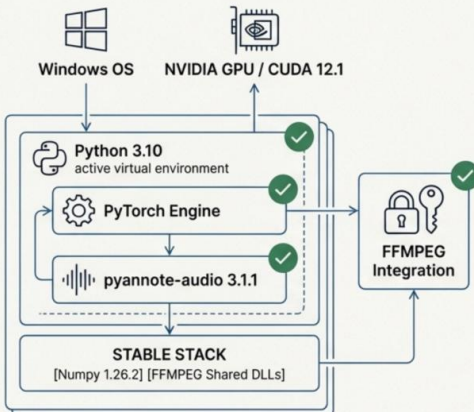
```

이 코드는 `os.add_dll_directory()` API 를 호출하여 윈도우 운영체제의 DLL 로더에게 "이 경로에 있는 DLL 들은 안전하니 로드해도 좋다"고 지시한다. 이 과정이 없으면, FFMPEG 이 시스템 PATH 에 있어도 Python 은 `FileNotFoundError` 또는 `ImportError: DLL load failed` 를 발생시킨다.

6. 통합 검증 및 벤치마킹

모든 설치 과정이 완료된 후, 환경의 적합성을 검증하는 절차가 필요하다. 다음은 전체 파이프라인(CUDA, PyTorch, FFMPEG, Pyannote)이 유기적으로 작동하는지 확인하는 검증 스크립트이다.

최종 시스템 검증: 모든 것이 올바르게 작동하는가?



아래 코드를 실행하여 GPU 가속, FFMPEG 연동, Pyannote 파이프라인 로딩이 모두 성공하는지 최종 확인하십시오.

```
Verification Script
D2Coding

import os
# [중요] FFMPEG DLL 로딩은 다른 라이브러리 일로부터 먼저 수행
try:
    os.add_dll_directory(r"C:\path\to\ffmpeg\bin") # 실제 경로로 수정
except Exception as e:
    print(f"DLL 로딩 경고: {e}")

import torch
from pyannote.audio import Pipeline

def verify_environment():
    print("== 환경 검증 시작 ==")
    # 1. CUDA 가속 확인
    cuda_available = torch.cuda.is_available()
    print(f"PyTorch 버전: {torch.__version__}")
    print(f"CUDA 사용 가능 여부: {cuda_available}")
    if cuda_available:
        print(f"GPU: {torch.cuda.get_device_name(0)}")
    else:
        print("경고: CPU 모드로 실행됩니다.")

    # 2. Pyannote 파이프라인 로드 테스트 (HuggingFace 토큰 필요)
    try:
        print("\nPyannote 파이프라인 로드 중...")
        pipeline = Pipeline.from_pretrained("pyannote/speaker-diarization-3.1")
        if cuda_available:
            pipeline.to(torch.device("cuda"))
        print("성공: 파이프라인이 메모리에 로드되었습니다.")
    except Exception as e:
        print(f"\n오류: 파이프라인 로드 실패: {e}")

if __name__ == "__main__":
    verify_environment()
```

예상 성능 및 리소스 사용량

이 환경(torch 2.x + cuda 12.1 + pyannote 3.1.1)이 정상적으로 구축되었을 경우, NVIDIA RTX 3060 급 GPU 기준으로 1 시간 분량의 오디오를 처리하는 데 약 **14 초에서 30 초** 소요될 것으로 예상된다. VRAM 사용량은 배치 처리에 따라 다르나, 일반적으로 4GB~6GB 수준에서 안정적으로 동작한다.

7. 결론

승리의 청사진: 성공을 보장한 3가지 핵심 전략



```
pip uninstall torchcodec
```

윈도우 DLL 충돌을 유발하는 잠재적 위험 요소를 **외과적으로 제거**하여 시스템의 근본적인 안정성을 확보했습니다.



```
numpy==1.26.2
```

최신 라이브러리 생태계의 불안정성으로 부터 시스템을 보호하기 위해, **ABI 호환성**이 검증된 버전을 **의도적으로 고정**했습니다.



```
os.add_dll_directory()
```

Conda의 도움 없이, 코드 레벨에서 FFMPEG DLL 경로를 **직접 주입**하여 윈도우의 보안 장벽을 넘어 멀티미디어 처리 능력을 활성화했습니다.

이 전략적 접근법은 단순한 설치를 넘어, 재현 가능하고 하드웨어 성능을 100% 활용하는 견고, 견고한 개발 환경을 구축합니다.

윈도우 환경에서 Conda 를 배제하고 Python 기본 도구만으로 고성능 화자 분리 환경을 구축하는 과정을 상세히 분석하였다. 단순한 설치 명령어의 나열을 넘어, 각 단계가 내포한 **Numpy ABI 호환성**, **윈도우 DLL 보안 모델**, **PyTorch/CUDA 버전 정합성** 문제를 기술적으로 해결하는 데 중점을 두었다.

특히 `pip uninstall torchcodec` 과 `os.add_dll_directory` 의 조합은 윈도우 Python 개발 환경에서 가장 까다로운 멀티미디어 의존성 문제를 해결하는 핵심 키(Key)이며, `numpy==1.26.2` 의 명시적 버전 고정은 최신 라이브러리 생태계의 불안정성으로부터 개발 환경을 보호하는 방패 역할을 한다. 이 구성을 통해 연구자 및 개발자는 재현 가능하고(Reproducible), 하드웨어 성능을 온전히 활용할 수 있는 견고한 개발 기반을 확보할 수 있다.

최종 확인 코드 : huggingface_whisper.ipynb

CELL IN 1

```
import torch
import sys
import transformers
from transformers import AutoProcessor

import os

import warnings
warnings.filterwarnings("ignore")

# -----
# 📁 경로를 새로 다운받은 "shared" 폴더의 bin 경로로 교체해주세요!
# (예시 경로이므로 본인의 실제 다운로드 경로로 수정 필수)
# -----
ffmpeg_path = r"C:\private_dev\LLM_App_dev\audio_test\lib\ffmpeg-7.1.1-
full_build-shared\bin"

if os.path.exists(ffmpeg_path):
    # 1. PATH 환경 변수 추가
    if ffmpeg_path not in os.environ.get('PATH', ''):
        os.environ['PATH'] = ffmpeg_path + os.pathsep +
os.environ.get('PATH', '')

    # 2. DLL 디렉토리 추가 (가장 중요!)
    if hasattr(os, 'add_dll_directory'):
        try:
            os.add_dll_directory(ffmpeg_path)
            print("☑ FFmpeg DLL 로딩 경로 설정 완료!")
        except Exception as e:
            print(f"⚠ 설정 중 에러: {e}")

def check_cuda():
    print(f"Transformers version: {transformers.__version__}")
    try:
        from transformers import AutoProcessor
        print("AutoProcessor imported successfully.")
    except ImportError as e:
        print(f"Error importing AutoProcessor: {e}")
```

```

try:
    from transformers.utils import add_model_info_to_auto_map
    print("Internal utils (add_model_info_to_auto_map) imported
successfully.")
except ImportError as e:
    print(f"Error importing internal utils: {e}")

print(f"Python version: {sys.version}")
print(f"PyTorch version: {torch.__version__}")

cuda_available = torch.cuda.is_available()
print(f"CUDA available: {cuda_available}")

if cuda_available:
    print(f"CUDA version: {torch.version.cuda}")
    device_count = torch.cuda.device_count()
    print(f"Device count: {device_count}")

    for i in range(device_count):
        print(f"Device {i}: {torch.cuda.get_device_name(i)}")
        print(f"  Capability: {torch.cuda.get_device_capability(i)}")

    current_device = torch.cuda.current_device()
    print(f"Current device index: {current_device}")
else:
    print("WARNING: CUDA is not available. PyTorch will run on CPU.")

check_cuda()

```

3.1 로컬 처리의 필요성 및 아키텍처

클라우드 API 는 편리하지만, 민감한 내부 정보를 다루는 회의나 보안 규정이 엄격한 금융/공공 기관에서는 외부 서버로 데이터를 전송하는 것이 불가능할 수 있습니다. 또한, 처리해야 할 오디오 데이터가 테라바이트(TB) 단위에 이를 경우, 고정 비용인 로컬 GPU 서버 구축이 변동 비용인 API 사용료보다 경제적일 수 있습니다.³

이러한 요구를 충족하기 위해 본 프로젝트에서는 허깅페이스(Hugging Face)의 transformers 라이브러리와 whisper-large-v3-turbo 모델을 활용한 온프레미스(On-Premise) 파이프라인을 구축합니다.

3.2 필수 전제 조건: FFmpeg 설치 및 환경 구성

로컬 접근법의 첫 관문: FFmpeg 환경 구성 마스터하기

파이썬 음성 처리 라이브러리의 보이지 않는 핵심 의존성 해결 가이드

The Problem

로컬 파이프라인 구축 시 가장 빈번한 실패 원인은 FFmpeg 의존성 문제입니다. 파이썬 라이브러리는 내부적으로 오디오 디코딩을 위해 시스템에 설치된 FFmpeg 바이너리를 호출하며, 경로 설정이 없으면 'FileNotFoundError'가 발생합니다.

The Solution: A Step-by-Step Guide

1.  Download: 'gyan.dev'에서 'ffmpeg-git-full.7z' 버전을 다운로드합니다. (Full 버전이 코덱 호환성이 높습니다.)	2.  Extract: 'C:\ffmpeg'와 같이 단순한 경로에 압축을 해제합니다.	3.  Set PATH (핵심 단계): '시스템 환경 변수 편집' -> '환경 변수' -> '시스템 변수'의 'Path'에 'C:\ffmpeg\bin' 경로를 추가합니다.	4.  Verify: CMD에서 'ffmpeg -version'을 실행하여 버전 정보가 출력되는지 확인합니다.
---	---	---	---

Pro-Tip

```
import os
# FFmpeg 바이너리 경로 명시적 추가
os.environ += os.pathsep + r"C:\ffmpeg\bin"
```

로컬 음성 처리 파이프라인 구축 시 가장 빈번하게 발생하는 오류는 **FFmpeg** 관련 문제입니다. 파이썬의 음성 처리 라이브러리들은 내부적으로 오디오 디코딩과 리샘플링을 위해 시스템에 설치된 FFmpeg 바이너리를 호출합니다. 이것이 없거나 경로가 설정되지 않으면 FileNotFoundError 또는 ValueError: ffmpeg was not found 와 같은 치명적인 오류가 발생합니다.

3.3 Whisper 파이프라인 구축 및 최적화

로컬 파이프라인 최적화: 메모리 관리와 정밀한 오디오 처리

양자화 (Quantization) - 더 적은 메모리로 더 빠르게

float32 → float16

- **Technique:** ``torch.float16`` 데이터 타입을 사용하여 모델 가중치를 반정밀도(Half-Precision)로 로드합니다.
- **Benefit:** 추론 정확도 손실은 최소화하면서 VRAM 사용량을 절반 가까이 줄이고 연산 속도를 가속화하는 핵심 기법입니다.

청크(Chunk) 처리 및 스트라이드(Stride) 전략

LONG AUDIO FILE

10초 Chunk, 10초 Chunk, 10초 Chunk

2초 Stride (Overlap)

Challenge: 트랜스포머 모델은 입력 길이에 제한(약 30초)이 있어 긴 오디오 파일을 작은 조각으로 나누어 처리해야 합니다.

파라미터	값	기술적 의미 및 근거
chunk_length_s	10초	오디오를 10초 단위로 분할하여 화자 교대 시 반응성을 높이고 메모리 부하를 분산시킵니다.
stride_length_s	2초	인접 청크 간 2초의 중복 구간을 두어, 경계면에서 단어가 잘리는 것을 방지하고 인식 정확도를 보장합니다 (Sliding Window 기법).

이 파이프라인의 결과물은 단순 텍스트가 아닌, 각 문장의 시작과 끝을 담은 정밀한 타임스탬프(Timestamp) 데이터를 포함합니다. 이 정보는 다음 단계의 핵심 입력이 됩니다.

로컬에서 whisper-large-v3-turbo 모델을 구동하기 위해서는 하드웨어 자원의 효율적인 관리가 필수적입니다.

3.3.1 양자화(Quantization)와 메모리 관리

대형 모델은 막대한 VRAM 을 소모합니다. 이를 해결하기 위해 `torch.float16` 데이터 타입을 사용하여 모델 가중치를 반정밀도(Half-Precision)로 로드합니다. 이는 추론 정확도의 손실은 최소화하면서 메모리 사용량을 절반 가까이 줄이고 연산 속도를 가속화하는 핵심 기법입니다.¹

3.3.2 청크(Chunk) 처리와 스트라이드(Stride) 전략

트랜스포머 기반 모델은 입력 길이에 제한이 있습니다(Whisper의 경우 일반적으로 30 초). 따라서 긴 회의 녹음 파일을 처리하기 위해서는 파일을 작은 조각(Chunk)으로 나누어 처리해야 합니다.

본 프로젝트에서는 `chunk_length_s=10` 과 `stride_length_s=2` 파라미터를 사용합니다.

파라미터	값	기술적 의미 및 근거
<code>chunk_length_s</code>	10 초	오디오를 10 초 단위로 분할하여 처리합니다. 30 초보다 짧게 설정함으로써 화자 교대 구간에서의 반응성을 높이고 메모리 부하를 분산시킵니다.
<code>stride_length_s</code>	2 초	인접한 청크 간에 2 초의 중복 구간을 둡니다. 이는 청크의 경계에서 단어가 잘리거나 문맥이 끊기는 것을 방지하여, 경계 구간의 인식 정확도를 보장하는 'Sliding Window' 기법의 일환입니다. ⁹

코드 : `huggingface_whisper.ipynb`

CELL IN 2

```
import torch

from transformers import AutoModelForSpeechSeq2Seq, AutoProcessor, pipeline

device = "cuda:0" if torch.cuda.is_available() else "cpu"
torch_dtype = torch.float16 if torch.cuda.is_available() else torch.float32

model_id = "openai/whisper-large-v3-turbo"
model = AutoModelForSpeechSeq2Seq.from_pretrained(
    model_id, torch_dtype=torch_dtype, low_cpu_mem_usage=True, use_safetensors=True
)
model.to(device)

processor = AutoProcessor.from_pretrained(model_id)
```

```

pipe = pipeline(
    "automatic-speech-recognition",
    model=model,
    tokenizer=processor.tokenizer,
    feature_extractor=processor.feature_extractor,
    torch_dtype=torch_dtype,
    device=device,
    return_timestamps=True, # 타임스탬프 반환 필수
    chunk_length_s=10,
    stride_length_s=2
)

```

이 파이프라인의 결과물은 전체 텍스트뿐 만 아니라, 각 문장이 언제 시작되고 끝나는지에 대한 정밀한 **타임스탬프(Timestamp)** 정보를 포함하게 됩니다. 이 정보는 후술할 화자 분리 데이터와의 정렬(Alignment)을 위해 필수적입니다.

코드 : huggingface_whisper.ipynb

CELL IN 3

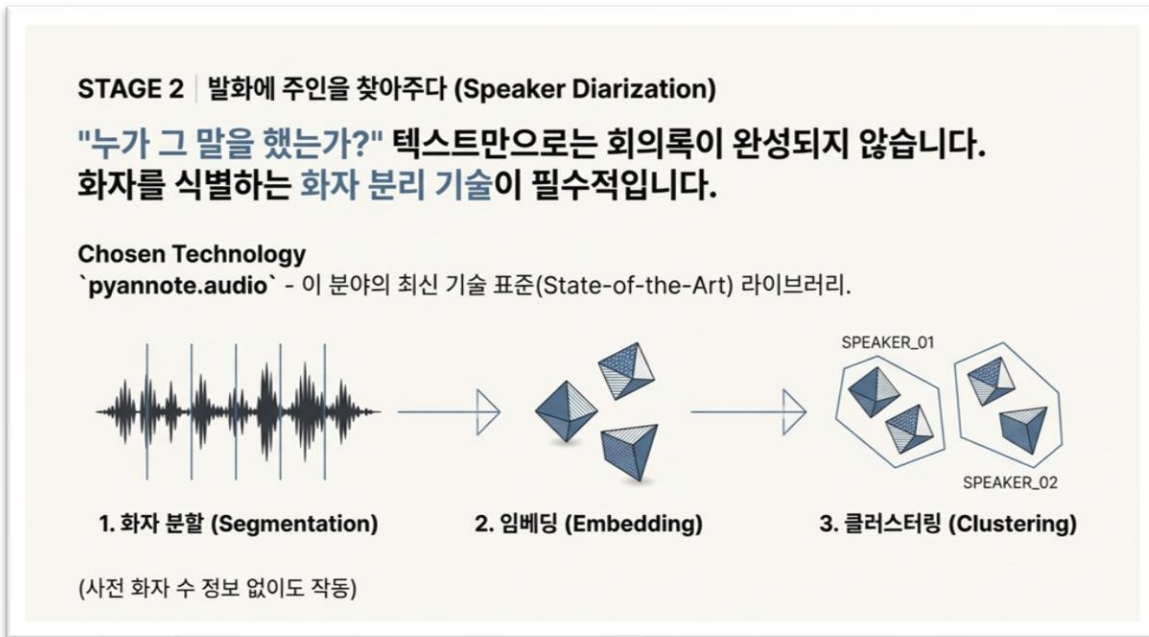
```

# chunks 를 CSV 파일로 저장
start_end_text = []
for chunk in result["chunks"]:
    start = chunk["timestamp"][0]
    end = chunk["timestamp"][1]
    text = chunk["text"]
    start_end_text.append([start, end, text])

import pandas as pd
df = pd.DataFrame(start_end_text, columns=["start", "end", "text"])
df.to_csv("lsy_audio_2023_58.csv", index=False, sep="|")
display(df)

```

4. 화자 분리(Speaker Diarization): "누가 말했는가?"



4.1 Pyannote.audio 아키텍처 분석

단순한 텍스트 변환만으로는 회의록을 완성할 수 없습니다. "누가" 그 말을 했는지를 식별하는 과정, 즉 **화자 분리(Speaker Diarization)**가 필요합니다. 본 프로젝트는 이 분야의 최신 기술 표준(State-of-the-Art)으로 평가받는 pyannote.audio 라이브러리를 사용합니다.

Pyannote.audio 파이프라인은 크게 세 단계로 작동합니다:

1. **화자 분할(Segmentation):** 오디오 스트림에서 화자가 바뀌는 지점을 순간적으로 감지합니다.
2. **임베딩(Embedding):** 분할된 각 오디오 조각을 고차원 벡터 공간으로 변환하여 목소리의 고유한 특성(지문)을 추출합니다.
3. **클러스터링(Clustering):** 추출된 벡터들을 분석하여 동일한 화자의 발화끼리 그룹화합니다. 이 과정은 사전에 화자 수(num_speakers)를 알지 못해도 작동하는 비지도 학습 방식을 지원합니다.

Consultant's Corner

의존성 지옥(Dependency Hell) 탈출하기: `pyannote.audio` 구현 시 반드시 마주치는 두 가지 함정과 해결책

Problem 1: Hugging Face Gated Model 인증

Symptom: `403 Forbidden` 또는 `GatedRepoError` 발생.

Cause: `pyannote/speaker-diarization-3.1` 모델은 사용 전 약관 동의가 필요한 Gated Model입니다.

Solution (Checklist):

- ☒ 허깅페이스 로그인
- ☒ 모델 페이지 방문하여 'Accept license' 클릭 (의존성 모델인 segmentation-3.0도 필수)
- ☒ 'Read' 권한의 Access Token 발급 및 코드에 주입

Problem 2: NumPy 버전 충돌 (가장 중요한 환경 설정 팁)

Symptom: `ImportError` 또는 `C-API mismatch` 오류.

Cause: `pyannote.audio`의 하위 라이브러리들이 NumPy 2.0+ 버전과 호환되지 않습니다.

Solution (The Silver Bullet): NumPy 버전을 1.26.x 대로 강제 고정(Downgrade)해야 합니다.

```
%pip install numpy==1.26
```

4.2 허깅페이스 Gated Model 인증 메커니즘(중요)

pyannote/speaker-diarization-3.1 모델은 오픈소스이지만, 무분별한 사용을 막고 사용자 동의를 받기 위해 **Gated Model** 정책을 취하고 있습니다. 따라서 단순한 코드 실행만으로는 모델을 다운로드할 수 없습니다.

필수 인증 절차:

1. 허깅페이스(huggingface.co) 웹사이트에 로그인합니다.
2. **pyannote/speaker-diarization-3.1** 모델 검색해서 페이지와 의존성 모델인 **pyannote/segmentation-3.0** 페이지를 각각 방문합니다.
3. 사용자 정보(소속, 웹사이트 등)를 입력하고 **약관 동의(Accept license)** 버튼을 클릭해야 합니다. 이 과정이 누락되면 403 Forbidden 또는 GatedRepoError 가 발생합니다.
4. **설정 페이지에서 'Read' 권한이 있는 Access Token** 을 발급받아 코드 내에 주입해야 합니다.

4.3 의존성 지옥(Dependency Hell): NumPy 버전 충돌 해결

본 프로젝트 구현 시 가장 주의해야 할 기술적 이슈는 **NumPy 라이브러리의 버전 호환성** 문제입니다. 최근 파이썬 생태계는 NumPy 2.0 출시로 인한 과도기를 겪고 있습니다.

- **문제 상황:** pyannote.audio 3.1 버전은 내부적으로 pyannote.core 와 pyannote.metrics 라이브러리에 의존합니다. 이 하위 라이브러리들 중 일부는 NumPy 2.0 이상의 최신 C-API 와

호환되지 않거나, 반대로 특정 버전 이하를 요구하는 등 복잡한 의존성 관계를 가지고 있습니다.¹²

- **증상:** 최신 환경에서 단순히 `pip install pyannote.audio` 를 실행하면, NumPy 2.x 가 설치되면서 ImportError 나 C-API mismatch 오류가 발생하여 실행이 불가능해집니다.
- **해결책:** 안정적인 구동을 위해 NumPy 버전을 **1.26.x** 대로 강제 고정(Downgrade)해야 합니다. 이는 본 프로젝트에서 제시하는 가장 중요한 환경 설정 팁 중 하나입니다.

```
%pip install numpy==1.26
```

4.4 RTTM 데이터 처리와 스무딩(Smoothing) 알고리즘

Diarization 모델의 출력 결과는 RTTM(Rich Transcription Time Marked) 형식으로 저장됩니다. 그러나 원본 모델의 출력은 매우 미세한 단위(예: 0.5 초)로 쪼개져 있는 경우가 많습니다.

스무딩 로직의 필요성:

동일한 화자가 10 초 동안 이야기하는 도중 숨을 쉬거나 잠시 멈추면, 모델은 이를 [0~4 초: 화자 A], [5~10 초: 화자 A]와 같이 분절된 형태로 출력할 수 있습니다. 이를 그대로 사용하면 회의록이 지나치게 파편화됩니다.

본 프로젝트에서는 판다스(Pandas)를 활용한 **병합 알고리즘**을 구현합니다 :

1. RTTM 데이터를 데이터프레임으로 로드합니다.
2. 연속된 행(Row)을 순회하며, 현재 행의 화자가 이전 행의 화자와 동일할 경우 동일한 그룹 번호를 부여합니다.
3. groupby 연산을 통해 동일 그룹의 시작 시간(min)과 종료 시간(max)을 계산하여 하나의 긴 발화 구간으로 병합합니다.

코드 : speaker_diarization.ipynb

CELL IN 1

```
import os  
  
from dotenv import load_dotenv  
  
load_dotenv()  
  
  
  
HUGGING_FACE_TOKEN = os.getenv("HUGGING_FACE_TOKEN")
```

CELL IN 2

```
# instantiate the pipeline
from pyannote.audio import Pipeline
import warnings
warnings.filterwarnings("ignore")

pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization-3.1",
    use_auth_token=HUGGING_FACE_TOKEN
)
```

CELL IN 3

```
import torch

# cuda 가 사용 가능한 경우 cuda 를 사용하도록 설정
if torch.cuda.is_available():
    pipeline.to(torch.device("cuda"))
    print('cuda is available')
else:
    print('cuda is not available')
```

CELL IN 4

```
# run the pipeline on an audio file
# diarization = pipeline("audio.wav")
diarization = pipeline("../audio/싼기타_비싼기타.mp3")

# dump the diarization output to disk using RTTM format
with open("싼기타_비싼기타.rttm", "w", encoding='utf-8') as rttm:
```

```
diarization.write_rttm(rttm)
```

CELL IN 5

```
# RTTM 을 CSV 로 변환
import pandas as pd
rttm_path = "./싼기타_비싼기타.rttm"

df_rttm = pd.read_csv(
    rttm_path,      # rttm 파일 경로
    sep=',',       # 구분자는 띄어쓰기
    header=None,    # 헤더는 없음
    names=['type', 'file', 'chnl', 'start', 'duration', 'C1', 'C2', 'speaker_id', 'C3', 'C4']
)

display(df_rttm)
```

CELL IN 6

```
# start + duration 을 end 로 변환
df_rttm['end'] = df_rttm['start'] + df_rttm['duration']

display(df_rttm)
```

CELL IN 7

```
df_rttm["number"] = None    # number 열 만들고 None 으로 초기화
df_rttm.at[0, "number"] = 0

display(df_rttm)
```

CELL IN 7

```
for i in range(1, len(df_rttm)):
    if df_rttm.at[i, "speaker_id"] != df_rttm.at[i-1, "speaker_id"]:
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"] + 1
    else:
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"]
display(df_rttm.head(10))
```

CELL IN 8

```
df_rttm_grouped = df_rttm.groupby("number").agg(
    start=pd.NamedAgg(column='start', aggfunc='min'),
    end=pd.NamedAgg(column='end', aggfunc='max'),
    speaker_id=pd.NamedAgg(column='speaker_id', aggfunc='first')
)
display(df_rttm_grouped)
```

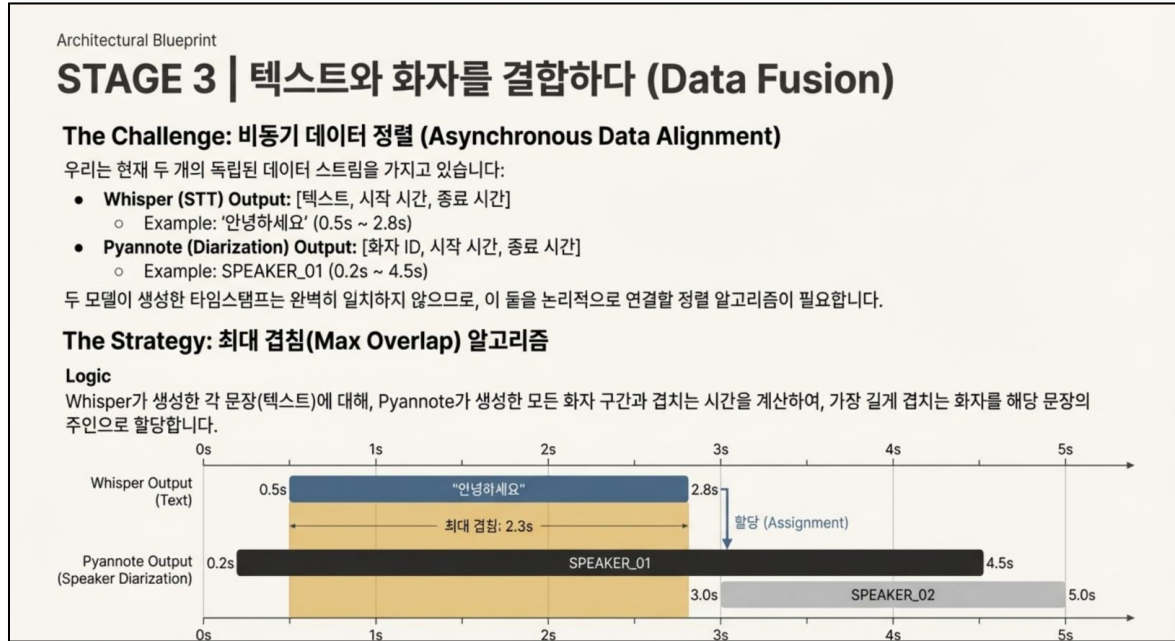
CELL IN 9

```
df_rttm_grouped["duration"] = df_rttm_grouped["end"] - df_rttm_grouped["start"]
df_rttm_grouped = df_rttm_grouped.reset_index(drop=True)
display(df_rttm_grouped)
```

CELL IN 10

```
df_rttm_grouped.to_csv(
    "싼기타_비싼기타_rttm.csv",
    sep=',',
    index=False
)
```

5. 데이터 융합(Data Fusion): 텍스트와 화자의 결합



5.1 비동기 데이터 정렬(Alignment) 문제

이제 우리는 두 가지 데이터 스트림을 가지고 있습니다:

1. **Whisper:** "안녕하세요" (00:00 ~ 00:03)
2. **Pyannote:** SPEAKER_01 (00:00 ~ 00:05)

이 두 데이터는 서로 다른 모델에서 생성되었으므로 타임스탬프가 완벽하게 일치하지 않습니다. 따라서 이를 합리적으로 매칭시켜주는 ****정렬 알고리즘(Matching Algorithm)****이 필요합니다.

5.2 교차 구간(Intersection) 기반 매칭 알고리즘

알고리즘 심층 분석: 교차 구간(Intersection) 기반 매칭

Algorithm Logic

1. Whisper가 생성한 각 문장 세그먼트 **S**에 대해 반복문을 실행합니다.
2. **S**의 시간 구간 (Start_S, End_S)과 Pyannote가 생성한 모든 화자 구간 **D** (Start_D, End_D)를 비교합니다.
3. 두 구간의 교집합, 즉 겹치는 시간의 길이(Overlap)를 계산합니다.
4. 가장 긴 Overlap을 가진 화자 구간 **D**의 화자 ID를 문장 **S**에 할당합니다.

$$\text{Overlap} = \text{Max}(0, \text{Min}(\text{End}_S, \text{End}_D) - \text{Max}(\text{Start}_S, \text{Start}_D))$$

```
# 정렬 알고리즘 핵심 로직
for i_stt, row_stt in df_stt.iterrows():
    overlap_dict = {}
    for i_rttm, row_rttm in df_rttm.iterrows():
        # 교차 구간 계산
        overlap = max(0, min(row_stt["end"], row_rttm["end"]) - max(row_stt["start"], row_rttm["start"]))
        overlap_dict[row_rttm["speaker"]] = overlap

# 가장 많이 겹치는 화자 선택
max_overlap_speaker = max(overlap_dict, key=overlap_dict.get)
# ... 화자 할당 ...
```

이 방식은 화자가 겹쳐 말하는 경우에도 가장 지배적인(dominant) 화자를 찾아내는 강건함(Robustness)을 가집니다.

본 프로젝트에서는 **최대 겹침(Max Overlap)** 전략을 사용합니다.¹

알고리즘 로직:

1. Whisper가 생성한 각 문장 세그먼트에 대해 반복문을 수행합니다.
2. 해당 문장의 시간 구간(Start~End)과 Pyannote가 생성한 모든 화자 구간을 비교합니다.
3. 두 구간의 교집합(Intersection), 즉 겹치는 시간의 길이를 계산합니다.
 - 수식: $\text{Overlap} = \text{Max}(0, \text{Min}(\text{End}_A, \text{End}_B) - \text{Max}(\text{Start}_A, \text{Start}_B))$
4. 가장 긴 시간 동안 겹치는 화자를 해당 문장의 주인으로 할당합니다.

소스 코드 : whisper_stt.ipynb

```
# 정렬 알고리즘 핵심 로직

for i_stt, row_stt in df_stt.iterrows():
    overlap_dict = {}
    for i_rttm, row_rttm in df_rttm.iterrows():
        # 교차 구간 계산
        overlap = max(0, min(row_stt["end"], row_rttm["end"]) - max(row_stt["start"],
row_rttm["start"]))
        overlap_dict[i_rttm] = overlap
```

```
# 가장 많이 겹치는 화자 선택
```

```
max_overlap_idx = max(overlap_dict, key=overlap_dict.get)
```

이 방식은 화자가 겹쳐서 말하거나(Overlapping Speech), 아주 짧은 추임새가 들어가는 경우에도 가장 지배적인 화자를 찾아내는 강건함(Robustness)을 보여줍니다.

전체 코드

CELL 1

```
import os

# -----
# 📁 경로를 새로 다운받은 "shared" 폴더의 bin 경로로 교체해주세요!
# (예시 경로이므로 본인의 실제 다운로드 경로로 수정 필수)
# -----

ffmpeg_path = r"C:\private_dev\LLM_App_dev\audio_test\lib\ffmpeg-7.1.1-full-build-
shared\bin"

if os.path.exists(ffmpeg_path):
    # 1. PATH 환경 변수 추가
    if ffmpeg_path not in os.environ.get('PATH', ''):
        os.environ['PATH'] = ffmpeg_path + os.pathsep + os.environ.get('PATH', '')

    # 2. DLL 디렉토리 추가 (가장 중요!)
    if hasattr(os, 'add_dll_directory'):
        try:
            os.add_dll_directory(ffmpeg_path)
            print("✅ FFmpeg DLL 로딩 경로 설정 완료!")
        except Exception as e:
            print(f"⚠️ 설정 중 에러: {e}")
```

```
import os
import torch
import pandas as pd
from transformers import AutoModelForSpeechSeq2Seq, AutoProcessor, pipeline
from pyannote.audio import Pipeline
import warnings
warnings.filterwarnings("ignore")

# -----
# 📁 경로를 새로 다운받은 "shared" 폴더의 bin 경로로 교체해주세요!
# (예시 경로이므로 본인의 실제 다운로드 경로로 수정 필수)
# -----

ffmpeg_path = r"C:\private_dev\LLM_App_dev\audio_test\lib\ffmpeg-7.1.1-full_build-shared\bin"

if os.path.exists(ffmpeg_path):
    # 1. PATH 환경 변수 추가
    if ffmpeg_path not in os.environ.get('PATH', ''):
        os.environ['PATH'] = ffmpeg_path + os.pathsep + os.environ.get('PATH', '')

    # 2. DLL 디렉토리 추가 (가장 중요!)
    if hasattr(os, 'add_dll_directory'):
        try:
            os.add_dll_directory(ffmpeg_path)
            print("☑ FFmpeg DLL 로딩 경로 설정 완료!")
        except Exception as e:
            print(f"⚠ 설정 중 에러: {e}")
```

```

def whisper_stt(
    audio_file_path: str,
    output_file_path: str = "./output.csv"
):
    device = "cuda:0" if torch.cuda.is_available() else "cpu"
    torch_dtype = torch.float16 if torch.cuda.is_available() else torch.float32
    model_id = "openai/whisper-large-v3-turbo"

    model = AutoModelForSpeechSeq2Seq.from_pretrained(
        model_id, torch_dtype=torch_dtype,
        low_cpu_mem_usage=True,
        use_safetensors=True
    )
    model.to(device)

    processor = AutoProcessor.from_pretrained(model_id)

    pipe = pipeline(
        "automatic-speech-recognition",
        model=model,
        tokenizer=processor.tokenizer,
        feature_extractor=processor.feature_extractor,
        dtype=torch_dtype,
        device=device,
        return_timestamps=True, # 청크별로 타임스탬프를 반환
        chunk_length_s=10, # 입력 오디오를 10 초씩 나누기
        stride_length_s=2, # 2 초씩 겹치도록 청크 나누기
    )

    result = pipe(audio_file_path)

```

```

df = whisper_to_dataframe(result, output_file_path)

return result, df

def whisper_to_dataframe(result, output_file_path):
    start_end_text = []

    for chunk in result["chunks"]:
        start = chunk["timestamp"][0]
        end = chunk["timestamp"][1]
        text = chunk["text"].strip()
        start_end_text.append([start, end, text])
        df = pd.DataFrame(start_end_text, columns=["start", "end", "text"])
        df.to_csv(output_file_path, index=False, sep="|")

    return df

def speaker_diarization(
    audio_file_path: str,
    output_rttm_file_path: str,
    output_csv_file_path: str
):
    pipeline = Pipeline.from_pretrained(
        "pyannote/speaker-diarization-3.1",
        use_auth_token="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"
    )

    # cuda 가 사용 가능한 경우 cuda 를 사용하도록 설정
    if torch.cuda.is_available():

```

```

    pipeline.to(torch.device("cuda"))
    print('cuda is available')
else:
    print('cuda is not available')
diarization_pipeline = pipeline(audio_file_path)

# dump the diarization output to disk using RTTM format
with open(output_rttm_file_path, "w", encoding='utf-8') as rttm:
    diarization_pipeline.write_rttm(rttm)

# pandas dataframe 으로 변환
df_rttm = pd.read_csv(
    output_rttm_file_path,    # rttm 파일 경로
    sep=' ',                # 구분자는 띄어쓰기
    header=None,            # 헤더는 없음
    names=['type', 'file', 'chnl', 'start', 'duration', 'C1', 'C2', 'speaker_id', 'C3', 'C4']
)

df_rttm["end"] = df_rttm["start"] + df_rttm["duration"]

# speaker_id 를 기반으로 화자별로 구간을 나누기
df_rttm["number"] = None
df_rttm.at[0, "number"] = 0

for i in range(1, len(df_rttm)):
    if df_rttm.at[i, "speaker_id"] != df_rttm.at[i-1, "speaker_id"]:
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"] + 1
    else:
        df_rttm.at[i, "number"] = df_rttm.at[i-1, "number"]

```

```

df_rttm_grouped = df_rttm.groupby("number").agg(
    start=pd.NamedAgg(column='start', aggfunc='min'),
    end=pd.NamedAgg(column='end', aggfunc='max'),
    speaker_id=pd.NamedAgg(column='speaker_id', aggfunc='first')
)

df_rttm_grouped["duration"] = df_rttm_grouped["end"] - df_rttm_grouped["start"]

df_rttm_grouped.to_csv(
    output_csv_file_path,
    index=False, # 인덱스는 저장하지 않음
    encoding='utf-8'
)

return df_rttm_grouped

def stt_to_rttm(
    audio_file_path: str,
    stt_output_file_path: str,
    rttm_file_path: str,
    rttm_csv_file_path: str,
    final_output_csv_file_path: str
):

    result, df_stt = whisper_stt(
        audio_file_path,
        stt_output_file_path
    ) #

    df_rttm = speaker_diarization(
        audio_file_path,

```

```

        rttm_file_path,
        rttm_csv_file_path
    ) #

df_rttm["text"] = "" #

for i_stt, row_stt in df_stt.iterrows(): #
    overlap_dict = {}
    for i_rttm, row_rttm in df_rttm.iterrows(): #
        overlap = max(0, min(row_stt["end"], row_rttm["end"]) - max(row_stt["start"],
row_rttm["start"]))
        overlap_dict[i_rttm] = overlap

    max_overlap = max(overlap_dict.values())
    max_overlap_idx = max(overlap_dict, key=overlap_dict.get)

    if max_overlap > 0: #
        df_rttm.at[max_overlap_idx, "text"] += row_stt["text"] + "\n"

df_rttm.to_csv(
    final_output_csv_file_path,
    index=False, # 인덱스는 저장하지 않음
    sep='|',
    encoding='utf-8'
) # ④

return df_rttm

# if __name__ == "__main__":
audio_file_path = "../audio/싼기타_비싼기타.mp3" # 원본 오디오 파일
stt_output_file_path = "싼기타_비싼기타.csv" # STT 결과 파일

```

```
rttm_file_path = "싼기타_비싼기타.rttm"          # 화자 분리 원본 파일
rttm_csv_file_path = "싼기타_비싼기타_rttm.csv" # 화자 분리 CSV 파일
final_csv_file_path = "싼기타_비싼기타_final.csv" # 최종 결과 파일

# result, df = whisper_stt(
#     audio_file_path,
#     stt_output_file_path
# )
# print(df)

# df_rttm = speaker_diarization(
#     audio_file_path,
#     rttm_file_path,
#     rttm_csv_file_path
# )

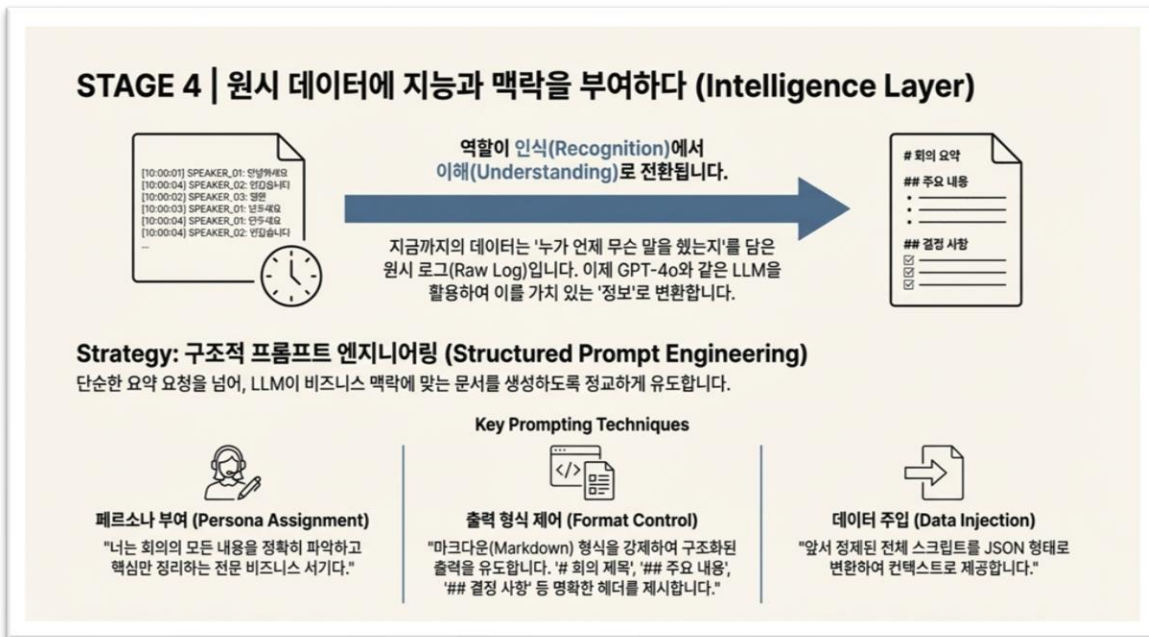
# print(df_rttm)

df_rttm = stt_to_rttm(
    audio_file_path,
    stt_output_file_path,
    rttm_file_path,
    rttm_csv_file_path,
    final_csv_file_path
)

print(df_rttm)
```

6. 인텔리전스 레이어: 요약 및 문맥 기반 교정

6.1 LLM의 역할 전환: 인식에서 이해로



정렬된 데이터는 이제 "누가 언제 무슨 말을 했는지"를 담고 있습니다. 하지만 이는 원시 데이터일 뿐, 회의록이라는 가치 있는 정보는 아닙니다. 이제 **GPT-4o**와 같은 거대 언어 모델(LLM)을 활용하여 데이터를 정제하고 가치를 부여합니다.

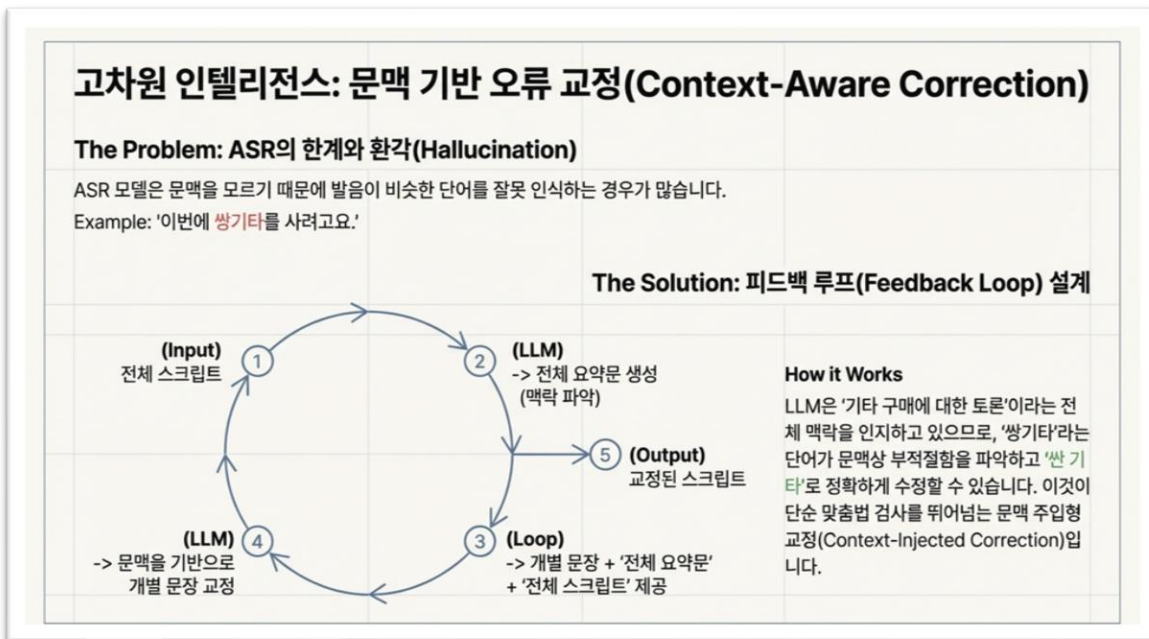
6.2 구조적 프롬프트 엔지니어링을 통한 요약

회의록 요약은 단순한 텍스트 압축이 아닙니다. 비즈니스 맥락에 맞는 구조화가 필요합니다. 본 프로젝트에서는 시스템 프롬프트(System Prompt)를 통해 LLM에게 ****전문 서기****라는 페르소나를 부여하고, 마크다운(Markdown) 형식을 강제합니다.

프롬프트 전략:

- **역할 정의:** "너는 회의 내용을 요약하는 봇이다."
- **출력 형식 제어:** #회의 제목, ##주요 내용, ##참석자별 입장, ##결정 사항 등의 헤더를 명시하여, LLM이 구조화된 문서를 생성하도록 유도합니다.
- **데이터 주입:** 정제된 전체 스크립트를 JSON 형태로 변환하여 컨텍스트로 제공합니다.

6.3 문맥 기반 오류 교정(Context-Aware Correction) 루프



ASR 모델은 발음이 비슷하거나 문맥을 모르면 엉뚱한 단어로 받아 적는 환각(Hallucination)이나 오류를 범할 수 있습니다. 예를 들어 "ChatGPT"를 "채 GPT"로, "싼 기타"를 "쌍기타"로 인식하는 경우입니다.

이를 해결하기 위해 본 프로젝트는 ****피드백 루프(Feedback Loop)****를 설계했습니다.

1. **전체 요약문 생성:** 먼저 회의 전체의 맥락을 파악하는 요약문을 생성합니다.
2. **개별 문장 교정 요청:** 각 문장을 순회하며 LLM에게 교정을 요청합니다. 이때, 단순히 문장만 주는 것이 아니라 **"전체 요약문"과 "전체 녹취록"을 함께 제공합니다.**
3. **추론:** LLM은 전체 맥락(기타 구매에 대한 토론)을 인지하고 있으므로, "쌍기타"라는 단어가 문맥상 맞지 않음을 파악하고 "싼 기타"로 수정할 수 있습니다.

이러한 ****문맥 주입형 교정(Context-Injected Correction)****은 단순한 맞춤법 검사기를 넘어선 고차원적인 지능형 처리 과정입니다.

7. 문서 생성 및 배포: 최종 산출물

7.1 자동화된 문서 포매팅

최종 단계는 사용자가 활용할 수 있는 형태의 파일로 결과를 출력하는 것입니다. 본 가이드는 마크다운(.md) 파일과 MS Word(.docx) 파일 생성을 모두 다룹니다.

7.2 python-docx 라이브러리 활용 심화

FINAL STAGE | 최종 산출물 생성 및 배포
자동화된 문서 포매팅: Markdown 및 MS Word(.docx)

Key Technology
`python-docx` 라이브러리를 활용한 자동 Word 문서 생성.

Consultant's Corner: `python-docx` 테이블 스타일링 이슈 해결

The Problem
`table.style = 'My Custom Style'`과 같이 코드를 작성하면, 해당 스타일이 문서의 내부 XML에 미리 정의되어 있지 않을 경우 에러가 발생하거나 스타일이 적용되지 않습니다.

The Safe Solution
`Table Grid`와 같은 Word의 기본 내장 스타일을 사용하는 것이 가장 안전하고 안정적입니다.

```
from docx import Document

document = Document()
table = document.add_table(rows=1, cols=2)
table.style = 'Table Grid' # 기본 그리드 스타일 적용
```

The Advanced Solution
커스텀 스타일이 반드시 필요하다면, 원하는 스타일이 미리 정의된 '템플릿.docx' 파일을 `Document('템플릿.docx')` 형태로 로드하여 사용하는 것이 기술적 모범 사례입니다.

MS Word 파일 생성 시 python-docx 라이브러리를 사용합니다. 여기서 주의할 점은 테이블 스타일링입니다.

테이블 스타일 이슈:

python-docx 에서 `table.style = 'Table Grid'`와 같이 스타일을 지정할 때, 해당 스타일이 문서의 내부 XML에 미리 정의되어 있지 않으면 에러가 발생하거나 스타일이 적용되지 않을 수 있습니다.

- **해결책:** 가장 안전한 방법은 'Table Grid'와 같은 기본 내장 스타일을 사용하는 것입니다. 만약 커스텀 스타일이 필요하다면, 미리 스타일이 정의된 '템플릿 docx' 파일을 로드(`Document('template.docx')`)하여 사용하는 것이 기술적 모범 사례입니다.

전체 코드 : summarize_and_correct.ipynb

IN 1

```
import pandas as pd

meeting_note_csv_path = "../sec03/싼기타_비싼기타_final.csv"
df_rttm = pd.read_csv(meeting_note_csv_path, sep='|')

display(df_rttm)
```

IN 2

```
# 이름 넣기
name_dict = {
    "SPEAKER_00": "AI",
    "SPEAKER_01": "해피데이",
}

df_rttm["name"] = df_rttm["speaker_id"].apply(lambda x: name_dict[x])

display(df_rttm)
```

IN 3

```
meeting_note_txt = df_rttm[["start", "end", "name", "text"]].to_json(orient='records',
force_ascii=False)
print(meeting_note_txt)
```

IN 4

```
system_prompt = f"""
너는 회의 내용을 요약하는 봇이다. 아래 회의록을 읽고, 주요 내용을 요약하라.
결과는 마크다운 형식으로 작성한다.

아래 형식에 맞추어 작성하라.

# 회의 제목

## 주요 내용

## 참석자별 입장

## 결정 사항

===== 이하 회의록 =====

{ meeting_note_txt }
"""
```

IN 5

```
from openai import OpenAI
from dotenv import load_dotenv
import os
```

```
load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
```

IN 6

```
# OpenAI API 를 사용해 요약 결과 생성
response = client.chat.completions.create(
    model='gpt-4o',
    temperature=0.1,
    messages=[
        {"role": "system", "content": system_prompt},
    ]
)

# (4) 요약 결과를 파일로 저장
summary = response.choices[0].message.content
summary = summary.strip() # 좌우 공백 제거

print(summary)
```

IN 7

```
with open('../audio/guitar_summary.md', 'w', encoding='utf-8') as f:
    f.write(summary)
```

IN 8

```
df_meeting_note = df_rttm[['start', 'end', 'name', 'text']].copy()
df_meeting_note.dropna(inplace=True)

meeting_note_dict = df_meeting_note.to_dict(orient='records') # 딕셔너리 형태로 변환
meeting_note_dict
```

IN 9

```
for row in meeting_note_dict:
    if not isinstance(row["text"], str): # 텍스트가 아닌 다른 형식인 경우 통과
        continue
    correction_system_prompt = f"""
    너는 주어진 회의 녹취록을 수정 및 보완하는 봇이다.
    주어진 회의 녹취록은 STT 로 작성된 결과이므로, 이 중에 오류가 있는 부분을 찾아내고
    수정하라.
```

표현, 말투는 최대한 원본과 일치하도록 유지하되, 잘못 받아적은 내용을 수정하라.

원본의 내용을 최대한 살리되, 잘못된 내용만 수정하라.

어떤 경우에는 A 라는 사람이 말한 내용을 B 가 말한 것으로 잘못 기록된 경우도 있을 수 있다. 이런 경우에는 문맥을 고려하여 수정하라.

원본 내용 중 빠지거나, 추가되거나, 잘못 기록된 부분을 찾아내어 수정하라. 수정된 메시지 이외에는 아무것도 작성하지 말라.

```
-----
회의 요약문 : {summary}
-----
```

```
회의 녹취록 전문: {df_meeting_note.to_json(orient='records', force_ascii=False)}
-----
```

```
확인할 메시지 원본: {row["text"]}
"""
```

```
response = client.chat.completions.create(
    model='gpt-4o',
    temperature=0.1,
    messages=[
        {"role": "system", "content": correction_system_prompt},
    ]
)
```

```
correction = response.choices[0].message.content
```

```
if ':' in correction: # “수정: “ 과 같은 형식으로 반환되는 경우 해결
    correction = correction.split(':')[1]
```

```
correction = correction.strip() # 좌우 공백 제거
```

```
print(row['name'])
print(row['text'])
print(correction)
print('-----')
```

```
row['corrected_text'] = correction
```

IN 10

```
# meeting_note_dict json 파일로 저장
import json

with open('../audio/guitar_meeting_note_corrected.json', 'w', encoding='utf-8') as f:
    json.dump(meeting_note_dict, f, ensure_ascii=False, indent=4)
```

IN 11

```
md_template = ""

for row in meeting_note_dict:
    md_template += f"- **{row['name']}** : {row['corrected_text']}\n"

with open('../audio/guitar_meeting_note_corrected.md', 'w', encoding='utf-8') as f:
    f.write(md_template)
```

IN 12

```
from docx import Document

document = Document() # docx 객체 생성
document.add_heading('회의록', level=1) # 제목 추가

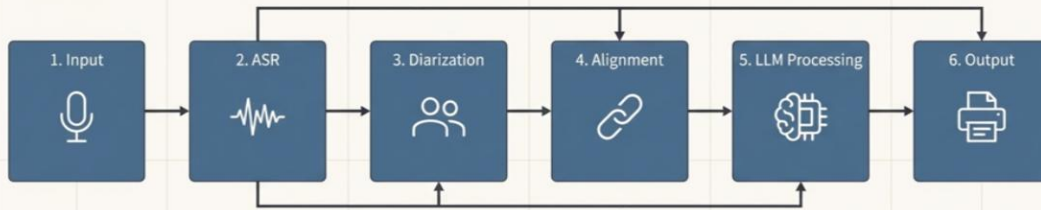
table = document.add_table(rows=1, cols=2) # 테이블 생성
table.style = 'Table Grid' # 테이블 스타일 지정

hdr_cells = table.rows[0].cells # 첫 번째 행에 셀 추가
hdr_cells[0].text = 'Speaker' # 첫 번째 셀에 Speaker 추가
hdr_cells[1].text = 'Content' # 두 번째 셀에 Content 추가

# 수정된 녹취록 파일을 읽어들이어서 테이블에 추가
for row in meeting_note_dict:
    row_cells = table.add_row().cells
    row_cells[0].text = row['name']
    row_cells[1].text = row['corrected_text']

document.save('../audio/guitar_meeting_note_corrected.docx') # (9) docx 파일 저장
```

완성된 청사진: 지능형 회의록 자동화 시스템



우리는 Whisper의 강력한 인식, Pyannote의 정교한 분리, GPT-4o의 문맥적 이해를 결합하여
단순 받아쓰기를 넘어선 '지능형 회의록' 파이프라인을 완성했습니다.
이 시스템은 기업의 디지털 전환(DX)을 가속화하는 실질적인 도구입니다.

이 가이드는 단순히 기술을 나열하는 것을 넘어, 각 모듈을 유기적으로 연결하고 실제 현장에서 발생하는 문제(환경
설정, 의존성, 데이터 정렬)를 해결하는 '엔지니어링의 청사진'을 제시했다는 점에서 차별화됩니다.